

# ThunderAgent: 简单、快速、程序感知的智能体推理系统

Hao Kang<sup>\*1</sup>, Ziyang Li<sup>\*2</sup>, Xinyu Yang<sup>\*3</sup>, Weili Xu<sup>\*4</sup>, Yinfang Chen<sup>4</sup>, Junxiong Wang<sup>5</sup>,  
Beidi Chen<sup>3</sup>, Tushar Krishna<sup>1</sup>, Chenfeng Xu<sup>5</sup>, and Simran Arora<sup>5</sup>

<sup>1</sup>Georgia Institute of Technology

<sup>2</sup>Individual Researcher

<sup>3</sup>Carnegie Mellon University

<sup>4</sup>University of Illinois Urbana-Champaign

<sup>5</sup>Together AI

[Code](#)

[Blog](#)

[Wiki](#)

## 摘要

大语言模型 (LLM) 如今被用于驱动复杂的多轮智能体 workflow。现有系统通常通过松散拼接相互隔离的组件来执行智能体推理: LLM 推理引擎 (如 vLLM) 和工具编排器 (如 Kubernetes)。尽管智能体 workflow 包含多次 LLM 请求和工具请求, 这些系统仍按请求分别调度与分配资源, 缺少对整个 workflow 的端到端认知, 导致 KV 缓存和工具执行环境管理不佳。为解决这些挑战, 我们提出 THUNDERAGENT, 一个快速、简单且程序感知的智能体推理系统。我们首先将智能体 workflow 抽象为 LLM 程序, 从而统一刻画 KV 缓存、系统状态以及磁盘空间、网络端口等外部工具资产。基于这一抽象, THUNDERAGENT 引入程序感知调度器和工具资源管理器, 以最大化 KV 缓存命中率、缓解显存不均衡, 并支持异步环境准备。跨代码、路由和科学发现智能体的评估表明, 相比当前最先进的推理系统, THUNDERAGENT 在服务吞吐上提升 **1.5-3.6** $\times$ , 在 RL rollout 上提升 **1.8-3.9** $\times$ , 并最多节省 **4.2** $\times$  磁盘空间。为便于复现和后续开发, 我们开源了完整的 THUNDERAGENT 系统实现: <https://github.com/HaoKang-Timmy/ThunderAgent>。

## 1 介绍

语言模型的最新进展已将其用途从基础聊天机器人扩展到复杂智能体 [20, 26]。这些智能体通过将长链推理与外部工具调用 (例如编译器、检索器) 交错起来, 解决编码 [7, 9] 和计算机使用 [2, 24] 等领域的现实问题, 并且通常作为无需实时人工干预即可执行多步骤 workflow 的自主系统运行。然而, 现代推理系统的吞吐量会随着待处理智能体请求数量增加而下降 (Figure 1a)。与此同时, rollout 占强化学习 (RL) 总挂钟时间的 **70%** 以上 [4, 17]。

随着智能体 workflow 在规模上越来越自治, 整体系统效率更多由持续吞吐量决定, 而不是像人机交互应用那样主要受尾延迟和用户响应时间支配。因此, 更高吞吐量可以让硬件服务更多已完成 workflow,

---

\* 共同一作。联系邮箱: [hkang342@gatech.edu](mailto:hkang342@gatech.edu)

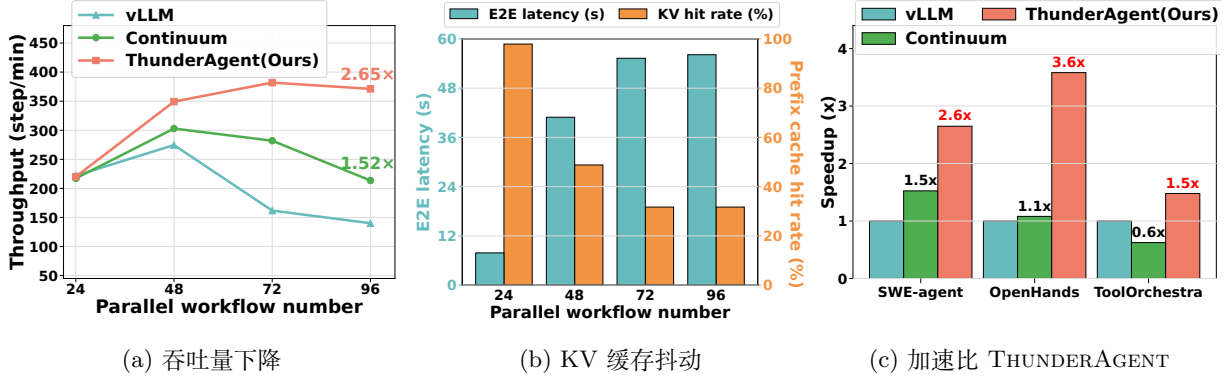


图 1: 随着并行工作流数量 (即批量大小) 增加, ThunderAgent 与既有智能体推理系统的性能比较。我们在 SWE-Bench Lite 上评估服务 SWE-Agent 的 GLM-4.6 MoE 模型 (图 a 和 b), 并在  $8 \times H100$  GPU 集群上评估 SWE-Agent、OpenHands 和 ToolOrchestra (图 c)。结果表明: (a) 当前推理系统无法在大批量下保持高吞吐量; (b) 吞吐量下降主要源于较低的 KV 缓存命中率, 这会增加端到端请求延迟; (c) 相比既有推理系统, THUNDERAGENT 通过减少 KV 缓存抖动并管理工具执行资源生命周期, 实现了更高吞吐量。

从而直接降低服务成本。此外, 在异步强化学习中, 更高的 rollout 吞吐量可以减轻数据收集所用参数与正在更新的参数之间的策略滞后, 使模型从陈旧性更低的数据中学习, 进而提升收敛速度和最终策略质量 [4, 16, 17, 32]。

然而, 当前智能体推理系统的吞吐量并不理想, 因为它们通常由彼此隔离的组件松散组合而成: 现成的模型推理引擎 (例如 vLLM [10] 或 SGLang [33]) 搭配通用工具编排器 (例如 Kubernetes)。虽然智能体工作流涉及多轮模型请求和工具请求, 这些组件仍在按请求的层面分别调度和分配资源, 缺少对整个工作流的端到端认知。这种设计带来了三个关键挑战:

1. **KV 缓存抖动**。请求感知系统在工具执行间隔期间过早地逐出 KV 缓存, 而没有预见到智能体工作流中的未来重用。因此, 当工具调用完成时, 系统需要重新运行预填充以恢复其整个交互历史记录。重新预填充成本会使智能体工作流的平均端到端延迟增加最多  $7.14\times$  (看 Figure 1b) 并降低吞吐量。
2. **跨节点显存不均衡**。请求感知引擎在多节点推理设置中存在利用率不平衡的问题。现有引擎将来自同一智能体工作流的所有请求固定到固定节点, 以最大限度地提高 KV 缓存命中率。然而, 由于上下文长度在智能体工作流中快速且不可预测地扩展, 在此路由策略下, 一些节点达到了容量, 而其他节点仍未得到充分利用。
3. **工具生命周期的遗忘**。请求感知的编排器很难决定何时释放和准备工具执行所需的资源和环境。因此, 未使用的沙箱和 API 服务器继续占用关键磁盘空间和网络端口, 导致资源累积耗尽和系统故障。同时, 智能体工作流在推理之前必须等待极长的设置时间。

本文介绍 THUNDERAGENT, 这是一种采用智能体工作流端到端视图的智能体推理系统, 用于实现高吞吐量智能体服务和 RL rollout。我们的具体贡献是:

1. **程序抽象**: 我们将智能体工作流抽象为智能体程序。智能体程序是一流的调度单元, 它在多个模型调用和工具执行中持续存在, 向运行时公开语义状态。程序跟踪工作流标识符、执行阶段 (即推理或

行动)、调度状态、总令牌和工具资源的元数据。这种抽象将调度与执行后端 (例如 vLLM/SGLang) 解耦, 从而实现新工作流的无缝集成。

2. **程序感知调度器**: 基于程序抽象, 我们将智能体推理调度视为约束优化问题, 以最小化重新计算和缓存开销, 并最大化预填充和解码吞吐量, 具体取决于 GPU 显存容量。我们引入两个关键机制:
  - (a) **状态感知暂停**: 如果执行后端遇到显存压力, 我们会通过工具调用选择性地暂停当前处于执行状态的工作流。这个设计有助于为处于推理状态的程序保留内存, 并消除任意的、次优的 KV 缓存驱逐。
  - (b) **动态迁移**: 我们跨数据并行 (DP) GPU 节点迁移智能体程序, 以减轻显存不均衡。我们通过以下方式实现这一点使所有 DP 节点共享一个全局程序感知等待队列, 而不是强制来自程序的请求始终发送到同一节点。
3. **程序感知工具资源管理**: 在长期智能体工作负载中, 工具环境是持久资源, 其管理不善直接限制持续吞吐量。通过跟踪执行依赖性, THUNDERAGENT 将 I/O 密集型环境初始化与 LLM 推理重叠。对于已完成的程序, 我们实现了生命周期感知的垃圾收集器利用程序终止信号来回收工具资源, 例如 Docker 沙箱和网络端口。因此, 这可以防止累积的资源泄漏并确保持续的高吞吐量智能体推理 THUNDERAGENT。

上述贡献无法在请求感知推理引擎中实现。如果没有程序状态和工作流依赖性的显式表示, 请求感知调度器无法区分临时工具等待和终止, 也无法将 GPU 显存与程序级资源调度协调起来。

我们评估 THUNDERAGENT 跨不同的智能体工作负载。为了 **服务**, 我们评估 ToolOrchestra [18] 作为 HLE-Bench 上的路由智能体 [15], SWE-Agent [27] 以及 OpenHands [23] 作为 SWE-bench 上的编码智能体 [9], 以及 OpenHands 作为 ScienceAgentBench 上的科学发现智能体 [3], 实现 **1.48–3.58×** 吞吐量改进如图所示 **Figure 1c**。为了 **强化学习 rollout**, 我们进一步在分布式 GPU 节点上测试编码智能体, 实现 **1.79–3.92×** 与之前的 SOTA 系统相比有所改进。

## 2 背景

在本节中, 我们提供支持智能体推理的属性和现有方法的背景。

### 2.1 当前智能体工作流的系统属性

当前的智能体工作流在生成过程中在推理和行动之间交替。正式地, 在每一步  $t$ , 智能体收到观察  $o_t \in \mathcal{O}$  并产生排放  $e_t = (\ell_t, a_t) \in \mathcal{L} \times \mathcal{A}$ , 在哪里  $\ell_t$  表示一个想法并且  $a_t$  代表一个动作。我们在步骤中定义累积上下文  $t$  作为  $c_t = (o_1, e_1, \dots, o_t)$ , 它捕获智能体工作流的交互历史记录。条件为  $c_t, e_t$  是从策略中采样的  $\pi(e_t|c_t)$ 。

此工作流保留两个持久状态: (i) GPU 显存, 其中 KV 缓存  $c_t$  充当工作流的内存。随着踪迹逐渐增长,  $c_{t+1}$  延伸  $c_t$  作为前缀, 实现理论上接近完整的 KV 缓存跨步骤重用率。(二) 工具环境, 其中外部资源 (例如沙箱或数据库连接) 初始化于  $t = 1$  在整个执行过程中必须保持一致且可访问。

这些有状态的依赖关系需要 程序级智能体推理轨迹的视图, 从而使系统能够协调异构资源并管理长期运行的工作流的状态。然而, 现有的推理系统对待每个想法  $\ell_t$  和行动  $a_t$  作为一个独立的、无状态的请求。

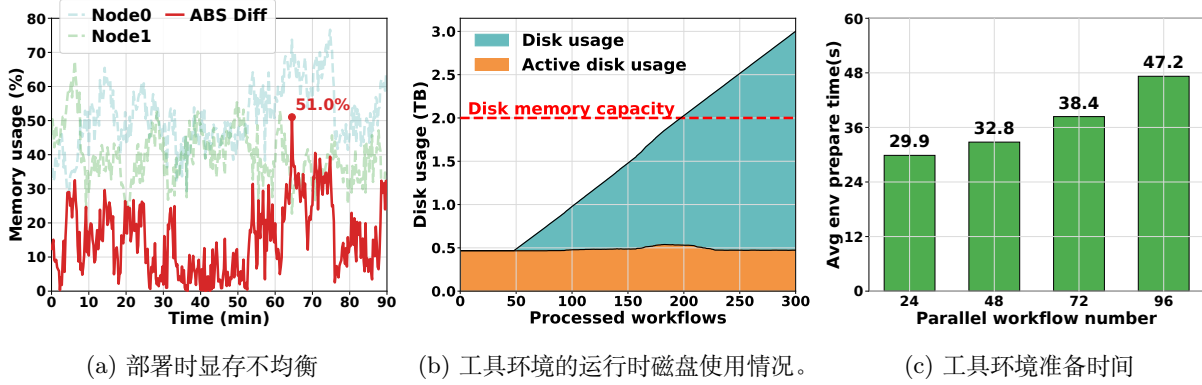


图 2: 当前智能体推理系统中的显存不均衡和工具资源管理问题。我们在 OpenHands RL rollout 上评估 vLLM + Kubernetes, 使用 SWE-Bench-Lite 上的 GLM-4.6 模型和两个 8×H100 GPU 节点。观察结果表明: (a) 应用 vLLM KV 感知路由器时, 在 90 分钟 rollout 测试中最大显存不均衡可达到 51%; (b) 工具执行环境垃圾回收失败会逐渐导致资源使用超过系统容量; (c) 随着并行工作流数量增加, 平均工具执行环境准备时间快速增长。

## 2.2 现有的智能体推理系统

之前的工作重点是优化智能体推理中的各个组件, 包括 LLM 推理引擎或工具编排器 (节 A.1, 节 A.2, 但很少有工作能够为跨 GPU、CPU 和远程资源的智能体工作流提供端到端优化。我们回顾这些先前的系统。

Autellix 将多轮智能体工作流建模为 **仅 GPU 程序**并在中央进程表中跟踪累积的 GPU 执行时间 [13]。然而, 它忽略了工作流局部性, 允许并发工作流积极驱逐其他工作流的 KV 缓存, 从而触发 **KV 缓存抖动**在繁重的工作负担下。

Continuum 是另一个最新的服务系统, 专为多轮智能体工作流而设计 [11]。它采用生存时间 (TTL) 机制将 KV 缓存固定在 HBM 中, 从而减轻工具执行期间的上下文抖动。然而, 它未能解决 KV 缓存驱逐问题。第一个原因是大多数工具都需要 **不可预测的时间量** (例如 ToolOrchestra 中的远程模型 API [18]、代码智能体中的编译器、以及计算机使用智能体中的网络应用程序 [35])。由于 TTL 估计不正确, 此类不可预测的工具会触发严重的抖动以及 Continuum 中的 KV 缓存内存搁浅。此外, 一旦正在运行的工作流的解码内存超过 GPU 限制, 系统也会抢占并驱逐固定的 KV 缓存。这会导致不可避免的抖动和相应的吞吐量下降, 如图所示 Figure 1a。

这些限制强调了对简单而快速的智能体推理系统的需求。我们设想这样一个系统作为新兴智能体推理系统的程序感知调度层 (例如, Zhang et al. [31])。

## 3 现有智能体推理系统面临的挑战

本节以 vLLM 与 Kubernetes 的组合作为多轮智能体推理的代表性基线, 并总结其中的关键低效问题。值得注意的是, 这些限制并不能仅通过替换推理引擎 (例如 TensorRT 或 SGLang) 或工具编排器解决, 而是需要新的程序感知抽象。默认设置下, 我们使用 GLM-4.6 模型, 在两个 8×H100 GPU 节点上运行 OpenHands RL rollout。



### 3.1 KV 缓存抖动

智能体工作流在执行期间具有较高的理论 KV 缓存重用率。然而，在现有 LLM 服务系统中，每一步都作为独立且无状态的请求处理。在高并发下，这种请求级调度会导致 KV 缓存在工具执行期间频繁被驱逐，以容纳新到达的请求；随之而来的重复驱逐和重新预填充就是 KV 缓存抖动。

如图所示 Figure 1b，随着并行工作流数量增加，这种抖动会加剧。由此导致的缓存命中率下降会触发频繁且成本高昂的重新预填充，系统必须在工具完成后重新计算整个历史记录。与非抖动设置相比，这种冗余会使每个请求的端到端延迟最高增加  $7.14\times$ ，并导致吞吐量严重下降。

### 3.2 跨节点显存不均衡

当前跨数据并行 (DP) 节点路由请求的策略也并非最优。现有多轮调度器 [21, 33] 会贪婪地把请求分配给 KV 缓存局部性最高的目标 DP 节点，以最大化缓存重用。然而，该策略忽略了节点之间显存负载可能变得不均衡这一事实。例如，vLLM 中的 KV 感知路由器 [21] 会将来自同一智能体工作流的所有请求发送到同一节点。由于不同工作流可能具有高度异构的 KV 占用和执行生命周期，这种策略常常导致节点之间严重显存不均衡：一些节点过载，而其他节点利用率较低。类似地，SGLang 中的前缀感知路由器会贪婪地把工作负载路由到具有匹配前缀的节点，以最大化缓存命中率。由于智能体系统提示在工作流中相同，该策略会把几乎所有请求发送到同一节点，同时让其他节点空闲。

如图所示 Figure 2a，在智能体 RL rollout 的 90 分钟快照中，两个 DP 节点之间的显存使用差距有超过 37 分钟 大于 20%，峰值不均衡达到 51%。

### 3.3 工具生命周期的遗忘

当前的智能体推理系统不将外部工具编排器的生命周期与 LLM 推理引擎同步，从而导致工具编排器端的无声资源浪费和延迟开销。

**资源泄漏和未使用的沙箱。** Figure 2b 展示了总磁盘空间占用随着处理的工作流数量线性增加，最终超过系统容量。这是因为工作流完成时未回收未使用的资源（例如已完成工作负载的 Docker 映像）。这种低效的垃圾收集会导致长期智能体推理的致命系统不稳定。

**昂贵的环境准备。** 我们观察到，大多数智能体工作负载需要在启动多轮轨迹之前准备环境。例如，编码智能体需要拉取 docker、安装相关包并构建存储库。此外，这种准备时间成本高昂，并且随着并行工作负载数量（即批量大小）的增加而增加，如图所示 Figure 2c。如果 LLM 推理引擎需要等到环境完全准备好，这种开销将延长推理系统的端到端延迟。

## 4 ThunderAgent：程序感知智能体推理系统

基于 节 3 中的发现，我们提出 THUNDERAGENT，一个用于高吞吐量智能体推理的程序感知系统。我们在 节 4.1 中建模智能体程序，这是系统调度的主要抽象。节 4.2 形式化了指导系统设计的成本模型。在这些基础上，我们进一步介绍 KV 缓存调度策略（节 4.3）和工具资源管理策略（节 4.4）。

表 1: **智能体程序符号摘要**。每个程序实例由其身份、执行阶段、工具环境、资源占用以及 THUNDER-AGENT 中的调度状态刻画。

| 符号            | 描述                                     |
|---------------|----------------------------------------|
| $P$           | 智能体程序实例                                |
| $ID$          | 程序的唯一全局标识符                             |
| $c$           | 上下文中的标记数量                              |
| $\mathcal{T}$ | 程序所需的工具环境集                             |
| $\mathcal{L}$ | 空间局部性的后端 (GPU 节点) 放置                   |
| $\tau$        | 执行阶段: 推理 ( <b>R</b> ), 执行 ( <b>A</b> ) |
| $s$           | 调度状态: {Active, Paused, Terminated}     |

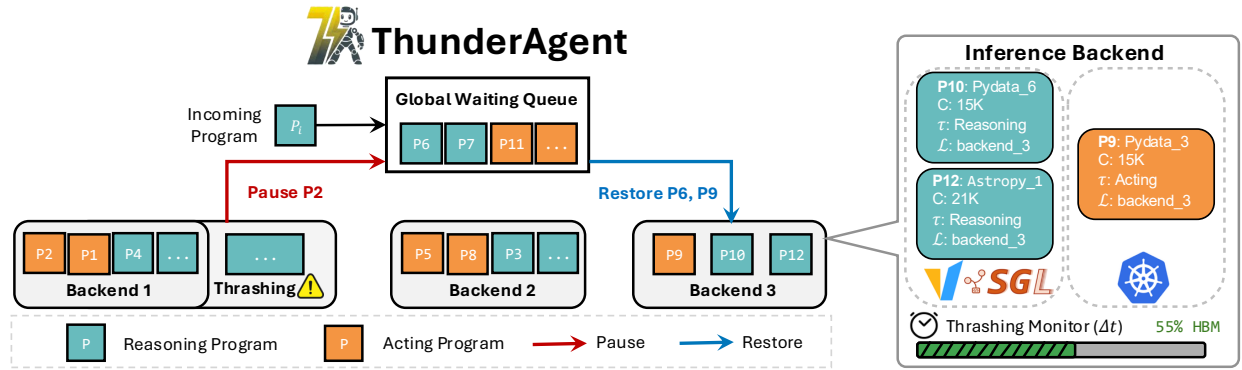


图 3: **ThunderAgent 概览**。图中展示了调度状态与显存管理之间的转换。THUNDERAGENT 每隔  $\Delta t$  定期查询各数据并行后端的状态。在这里，后端 #1 触发动抖，而后端 #3 利用不足。所有后端共享的全局等待队列会暂停并回收处于执行阶段的程序 #2，同时释放推理阶段程序 #6 和 #9，以停止后端 #1 中的 KV 缓存抖动，并降低后端 #3 的显存不均衡。

#### 4.1 程序抽象

**智能体程序**是一种基础抽象，用于封装智能体工作流的逻辑执行流和系统级依赖。形式化地，我们将智能体程序  $P$  定义为如下元组：

$$P = \langle ID, c, \mathcal{T}, \mathcal{L}, \tau, s \rangle, \quad (1)$$

其中， $ID$  表示唯一全局标识符。 $c$  表示上下文中的令牌数量，对应主动执行期间的 KV 缓存显存占用。 $\mathcal{T}$  跟踪程序使用的工具环境集合，使系统能在没有程序继续需要这些环境时进行垃圾回收。 $\mathcal{L}$ 、 $\tau$  和  $s$  分别表示节点放置、执行阶段和调度状态，从而支持程序级 KV 缓存抖动缓解和跨节点迁移。Figure 3 右侧给出了一个元数据示例。

THUNDERAGENT 通过对接 OpenAI 风格端点，直接封装现有 LLM 引擎和工具编排器。程序 ID 允许系统区分来自不同智能体工作流的请求。我们在附录 B 中详细说明 THUNDERAGENT 与现有推理服务集成的简洁性。

## 4.2 成本模型

在多轮智能体推理过程中，只有用于主动预填充和解码的资源才有助于系统的有效吞吐量，而重新计算、已用容量和空闲缓存则构成资源浪费。我们将其包含在 GPU 资源消耗的成本模型中，该模型将有效成本与非生产性使用隔离开来。我们采用时空积 (STP) [1] 作为我们的主要指标，定义为内存占用量随处理时间的积分。流程阶段的 STP 成本形式化为：

$$\text{Cost}_x = \int_0^{t_x} M_x(t) dt, \quad (2)$$

在哪里  $t_x$  是过程的持续时间  $x$ （例如，预填充）。由于内存使用  $M_x(t)$  可以通过 LLM 中使用的 KV 缓存令牌计数直接量化，我们将成本模型定义为令牌计数随时间的积分。

智能体推理的总成本包括五个不同的部分：解码、预填充、重新计算、未使用的容量和空闲缓存。我们明确区分工具执行结果的增量预填充和历史交互的重新计算，后者由于在整个上下文上重新计算被逐出的 KV 缓存而导致成本显著升高。形式上，这会产生以下成本分解：

$$\text{Cost}_{\text{total}} \approx \text{Cost}_{\text{decode}} + \text{Cost}_{\text{prefill}} + \text{Cost}_{\text{recompute}} + \text{Cost}_{\text{unused}} + \text{Cost}_{\text{caching}} \quad (3)$$

在这个分解中， $\text{Cost}_{\text{decode}}$  和  $\text{Cost}_{\text{prefill}}$  代表有助于推理吞吐量的有效工作。其余的项都是浪费的系统开销： $\text{Cost}_{\text{recompute}}$  源于 KV 缓存抖动 (节 3.1)； $\text{Cost}_{\text{unused}}$  反映数据并行 (DP) 推理后端副本之间的显存不均衡 (节 3.2)；和  $\text{Cost}_{\text{caching}}$  在外部工具执行期间保持内存的同时进行累加 (节 3.3)。

## 4.3 调度策略

基于上面的成本模型，我们的调度策略的优化目标是最小化非生产性开销部分： $\text{Cost}_{\text{recompute}}$ ， $\text{Cost}_{\text{unused}}$ ，和  $\text{Cost}_{\text{caching}}$ ，从而最大化吞吐量。

### 4.3.1 通过程序感知等待队列减少重新计算和缓存成本

正如所确定的 节 3.1 和 Figure 1b，KV 缓存抖动是吞吐量下降的主要瓶颈。为了解决这个限制，系统必须最小化  $\text{Cost}_{\text{recompute}}$  通过明确控制活动程序的数量。THUNDERAGENT 通过引入程序感知的等待队列来实现这一点。我们的系统利用这个队列来调度器执行，根据令牌长度确定哪些程序应该在 GPU 中执行，哪些程序应该被换出  $c$  和执行阶段  $\tau$ 。在这里，我们使用两个原始操作来形式化调度器行为：**恢复**和**暂停**，如下。

- **恢复**。此操作允许程序进入活动执行状态。给定一个程序  $P = \langle ID, c, \mathcal{T}, \mathcal{L}, \tau, s \rangle$  和  $s = \text{Paused}$  和  $\mathcal{L} = \emptyset$ ，恢复 ( $P$ ) 分配  $P$  到后端  $\mathcal{L}'$  具有可用容量和更新

$$P \leftarrow \langle ID, c, \mathcal{T}, \mathcal{L}', \tau, \text{Active} \rangle, \quad (4)$$

- **暂停**。此操作将程序从活动执行中删除。给定一个程序  $P = \langle ID, c, \mathcal{T}, \mathcal{L}, \tau, s \rangle$  和  $s = \text{Active}$ ，暂停 ( $P$ ) 解除绑定  $P$  从其后端释放其 KV 缓存以进行抢占，并更新

$$P \leftarrow \langle ID, c, \mathcal{T}, \emptyset, \tau, \text{Paused} \rangle. \quad (5)$$

基于这两个操作，我们接下来介绍我们的调度策略，以最大限度地减少 KV 缓存抖动。

**定期抖动检测。** 程序抽象为 节 4.1 为我们提供了智能体程序的 KV 缓存大小。值得注意的是，这在请求级系统中不可用（如 节 3）。我们定义 DP 后端的抖动条件  $\mathcal{L}$  当程序内存需求超过总容量时：

$$C_{\text{total}} < \sum_{p \in \mathcal{L}} c_p \quad (6)$$

在哪里  $C_{\text{total}}$  表示后端 KV 缓存池的固定 token 容量  $\mathcal{L}$ 。在解码过程中，上下文长度  $c_p$  智能体工作流的数量快速增长，这可能会引发内存抖动 执行中期即使没有新来者。与仅在工作流到达时执行是否接纳工作流的检查的基线调度器（例如，Continuum）不同，我们实现了 **定期监测**以固定时间间隔评估内存使用情况  $\Delta t$ ，允许抢先检测并减轻上下文增长引起的显存压力。

当 KV 缓存抖动即将发生时，THUNDERAGENT 调用 **暂停**暂停活动程序和释放内存大小的操作  $\Delta C = \sum_{p \in \mathcal{L}} c_p - \lambda_{\max} \cdot C_{\text{total}}$  直到总内存使用量低于限制  $\lambda_{\max} \cdot C_{\text{total}}$ 。相反，当后端有可用空间时，意味着  $\sum_{p \in \mathcal{L}} c_p < \lambda_{\min} \cdot C_{\text{total}}$ ，THUNDERAGENT 通过以下方式从等待队列中恢复暂停的程序 **恢复**，确保恢复的程序保持总内存低于  $\lambda_{\max} \cdot C_{\text{total}}$ 。这里， $\lambda_{\max}$  和  $\lambda_{\min}$  表示高水位线和低水位线的内存使用情况，分别共同形成一个滞后窗口稳定我们的日程安排。在实践中，我们将这两个值都设置为 1，因为跨程序的共享提示隐式保留了足够的内存缓冲区。

通过此程序级定期容量检查，THUNDERAGENT 可以通过在执行阶段为活动程序保留内存来保证不会出现 KV 缓存抖动。然而，代价是当程序进行长时间运行的工具执行时，执行程序占用的 GPU 显存处于空闲状态。为了平衡缓存与重新计算的成本，我们将时间衰减机制纳入抖动检查中，逐步降低执行程序令牌的有效权重。这允许调度器在显存压力上升时驱逐长时间空闲的缓存，而不是无限期地保留它们：

$$C_{\text{total}} < \sum_{p \in \mathcal{L}, \tau = \mathbf{R}} c_p + \sum_{q \in \mathcal{L}, \tau = \mathbf{A}} c_q \times f(t_q) \quad (7)$$

具体来说， $t_q$  是程序的工具执行时间  $q$  在当前步骤中。 $f(t)$  是一个时间衰减函数，旨在平衡  $\text{Cost}_{\text{caching}}$  和  $\text{Cost}_{\text{recompute}}$ 。通过随着时间的推移动态降低正在执行的程序的有效内存优先级， $f(t)$  鼓励调度器驱逐保持空闲的缓存。在 节 E.1，我们证明当工具执行延迟满足无记忆属性（即剩余执行时间与经过的持续时间无关）时，最优衰减函数  $f(t)$  采取指数衰减的形式。

**最小化  $\text{Cost}_{\text{recompute}}$  通过最短优先驱逐。** 有了上述驱逐和恢复条件，处理抖动的剩余问题是确定暂停活动程序的哪个子集，以使重新计算成本最小化。在本段中，我们证明了驱逐具有最小 KV 缓存大小的程序会产生最佳解决方案，详细证明参见 节 E.2。

**Lemma 4.1** (Quadratic Recomputation Cost). 给定一个程序  $P_i$  与上下文长度  $c_i$ ，重新填充其 KV 缓存所产生的重新计算成本与  $c_i$ ， $IE$ ，

$$\text{Cost}_{\text{recompute}} = \int_0^{t_{\text{recompute}}} c_i(t) dt \propto c_i^2. \quad (8)$$

**Definition 4.1** (Eviction Optimization Problem). 基于引理 4.1，给定所需的内存释放  $\Delta C$ ，调度器的目标是选择一个子集  $S$  的程序驱逐，以便释放容量满足约束，同时最小化总容量重新计算成本。该优化问题表述如下：

$$\min_S \sum_{i \in S} c_i^2 \quad \text{s.t.} \quad \sum_{i \in S} c_i \geq \Delta C. \quad (9)$$



目标通过选择较小的  $c_i$  得到严格最小化。因此，THUNDERAGENT 的策略是贪婪地暂停并驱逐具有 **最短上下文长度** 的程序。形式化证明见附录 E.3。基于这些分析，我们在调度器中使用以下分数来恢复和暂停程序：

$$S_{\text{restore}}(P) = \frac{1}{c_P} + \mathbb{I}(\tau = \mathbf{R}) \quad (10)$$

$$S_{\text{pause}}(P) = \frac{1}{c_P} + \mathbb{I}(\tau = \mathbf{A}) \quad (11)$$

其中指标函数  $\mathbb{I}(\cdot)$  强制执行程序执行状态的严格优先级 ( $\tau$ ) 超过上下文长度。两种机制都遵循 **最短优先** 最小化重新计算成本的策略。然而，状态指示器  $\mathbb{I}$  确保调度器优先考虑暂停智能体程序，从而最大限度地减少  $\text{Cost}_{\text{caching}}$  通过回收缓存内存，同时优先考虑恢复推理程序以最大化  $\text{Cost}_{\text{decode}} + \text{Cost}_{\text{prefill}}$ 。

#### 4.3.2 通过全局程序感知等待队列减少显存不均衡

节 3.1 和 Figure 2a 强调节点之间的显存不均衡会带来显著的影响  $\text{Cost}_{\text{unused}}$ ，尽管其他节点有足够的 GPU 显存容量，但仍会导致不必要的程序暂停。为此，THUNDERAGENT 将所有后端副本的等待队列统一为一个 **全局程序感知等待队列**。

这个设计的关键动机是  $\text{Cost}_{\text{unused}}$  仅出现当暂停的程序仍在等待队列中，而某些副本有闲置内存。而且，一旦程序暂停，它的 KV 缓存就会被假设被驱逐，使其重新计算成本与节点无关。这使我们能够在不牺牲 KV 缓存局部性的情况下改善跨节点内存平衡。恢复策略与负载均衡相一致，而不是严格的 KV 感知路由，使暂停的程序能够调度到任何具有可用 GPU 显存容量的副本。因此，全局队列限制了未使用的成本，使得  $C_{\text{unused}} < c_{\min} \cdot \Delta t^1$  对于周期内的每个节点  $\Delta t$ ，在哪里  $c_{\min}$  表示暂停程序中的最小令牌长度。调度策略和全局等待队列概述 THUNDERAGENT 提出于 Figure 3。

### 4.4 工具资源管理

下一个，THUNDERAGENT 减轻资源泄漏和环境设置开销，详见 节 3.3。

**基于钩子的垃圾收集。** 我们实现了生命周期钩子，将工具资源的持久性与智能体程序的调度状态严格耦合起来  $s$ 。当一个程序是终止，收集器立即触发拆卸序列，系统地回收沙箱、网络套接字和计算槽。活动磁盘使用情况 Figure 2b 展示了我们的资源管理策略有效地防止了过多资源的积累，随着时间的推移保持了近乎恒定的磁盘空间占用。

**异步环境准备。** 初始化工具执行环境所涉及的延迟（例如，启动 Docker 容器并安装依赖项）可能是瓶颈。为了解决这个问题，THUNDERAGENT 监控全局等待队列；当高优先级程序（高  $S_{\text{restore}}$ ）接近恢复阈值时，系统会在分配 GPU 显存之前异步恢复其执行环境。这项技术有效地 **隐藏初始化开销**，显著减少工具调用繁重工作负载（例如编码智能体和科学智能体）的端到端延迟，如 Figure 2c。

## 5 实验

在本节中，我们在多种智能体工作流上评估 THUNDERAGENT，包括编码、路由和科学研究智能体，以及从 RTX 5090 到 H100 集群的多种硬件配置下的 RL rollout。此外，我们进行了广泛的消融研

<sup>1</sup> 自从  $\Delta t$  比程序的生命周期小得多，我们忽略单个时间间隔内终止程序的影响。

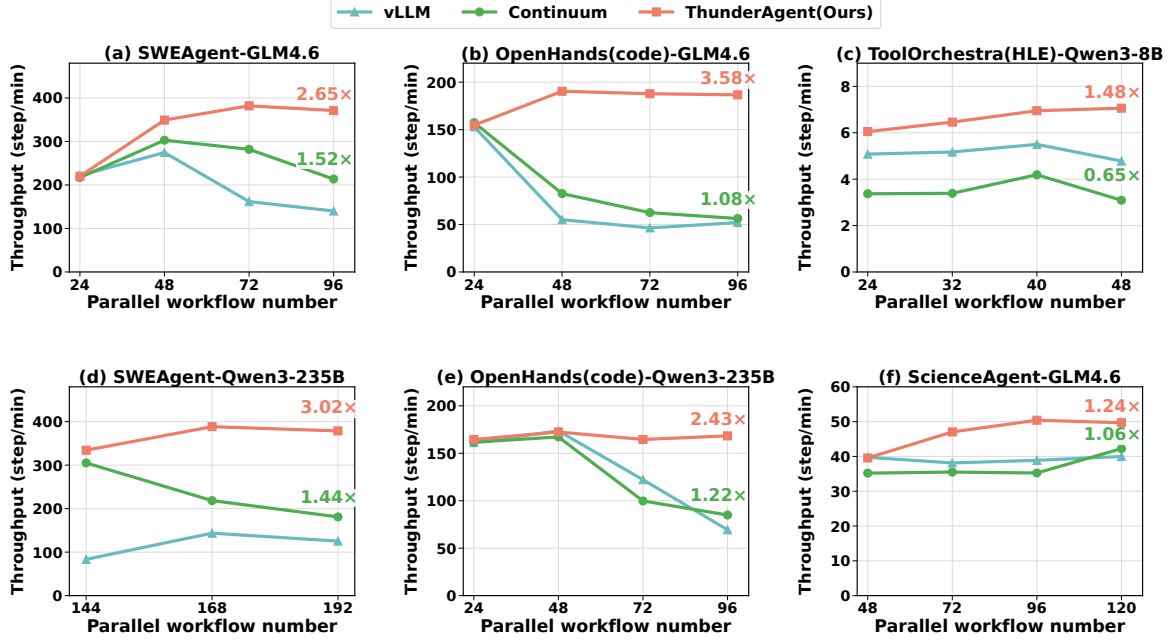


图 4: 服务评估结果。THUNDERAGENT 在三个模型、四个智能体工作流和三个数据集上, 其性能显著优于 vLLM 和 Continuum。对于具有可预测工具调用时间的工作流 (例如, a、b、d、e), THUNDERAGENT 优于 vLLM 和 Continuum, 高达 2.43-3.56 $\times$ 。对于具有随机工具执行时间的工作流 (例如, c、f), THUNDERAGENT 仍然达到最佳的吞吐性能。

究, 用于分解端到端系统运行时间, 并在 节 5.4 中分析系统对调度器超参数  $\Delta t$  和  $f(t)$  的敏感性。

## 5.1 实验装置

**基准和工作流。** 我们评估 THUNDERAGENT 针对不同的基准和工作负载:

1. **编码智能体服务。** 我们在 SWE-Bench-Lite [9] 数据集上部署 OpenHands 和 mini-SWEAgent。OpenHands 代表 **重初始化工作流**, 每个沙箱的平均磁盘占用超过 10GB; mini-SWEAgent 则代表 **轻量级工作流**, 占用较小 ( $\approx 2$ GB)。
2. **其他智能体服务。** 我们在 HLE [15] 上应用 ToolOrchestra, 并在 ScienceAgentBench [3] 上应用 OpenHands。这些工作负载涉及外部 API 调用和复杂科学模拟所带来的可变延迟。
3. **RL rollout。** 我们在两个  $8 \times H100$  节点上使用相同模型、工作流和样本进行 RL rollout。

**模型和部署。** 对于 OpenHands [23] 和 mini-SWEAgent [27] 框架, 我们采用 GLM-4.6 (355B) [20] 和 Qwen-3 (235B) [26]。模型被量化为 FP8, 并在单个  $8 \times H100$  节点上使用张量并行 (TP=8)。对于 ToolOrchestra [18], 我们使用托管在一台 RTX 5090 上、FP16 精度的 Qwen3-8B。我们将 LLM 推理引擎和 Docker 部署在不同集群上: LLM 推理引擎运行在承载模型的 GPU 集群上, 智能体 Docker 环境则卸载到专用 CPU 集群。

**ThunderAgent 配置。** 我们配置 THUNDERAGENT 带超参数  $\Delta t = 5$  和优先级衰减  $f(t) = 2^{-t}$ , 定义于 节 4.3。vLLM 被用作我们的 LLM 推理引擎。我们使用每分钟的步骤作为吞吐量指标, 其中一个步骤包括工作流的推理和执行周期。

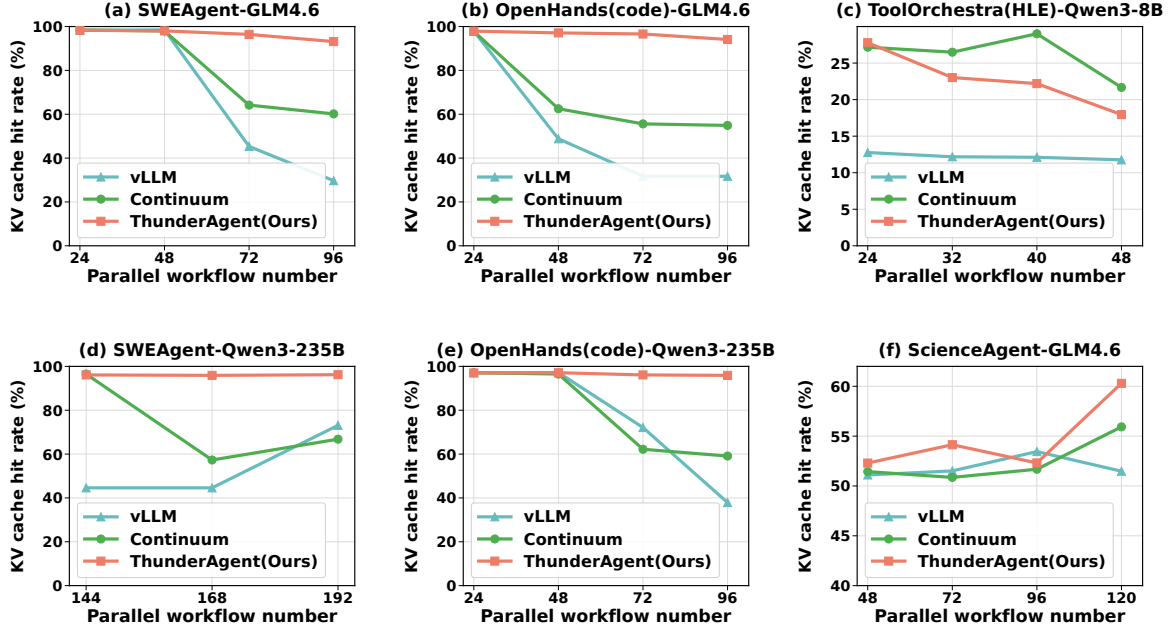


图 5: KV 缓存命中率统计。对于工具调用时间可预测的工作流 (a、b、d、e), THUNDERAGENT 达到接近最优 ( $\approx 100\%$ ) 的命中率; 对于工具执行时间随机的工作流 (c、f), THUNDERAGENT 会动态权衡命中率以减少空闲缓存。与 vLLM 和 Continuum 相比, 它总体上实现了更高的 KV 缓存命中率。

**基线技术。** 我们与具有不同调度范例的最先进系统进行比较:

- **vLLM (推理):** 一种广泛采用的请求感知型 LLM 推理引擎, 可作为推理性能的无状态基线, 无需结合任何特定于智能体或程序的感知。
- **Continuum (推理):** 当前用于多轮智能体工作流的 SOTA 系统。它通过预测工具执行持续时间并将 KV 缓存相应地固定到 HBM 来减轻 KV 缓存抖动。
- **vLLM + SGLang Gateway (分布式 rollout):** 大规模分布式强化学习 rollout 的领先解决方案。SGLang Gateway 通过增强跨节点显存平衡和 KV 缓存命中率来优化分布式推理, 使这种组合成为分布式 RL rollout 设置下的强大基线。

## 5.2 服务评估结果

**高并发下的高吞吐量。** Figure 4 表明, THUNDERAGENT 在高并发级别 (例如 96 个并行程序) 下具有显著吞吐优势: 在不同基础模型和数据集上, 相比 vLLM 实现  $1.48\text{--}3.58\times$  加速, 相比 Continuum 实现  $1.17\text{--}3.31\times$  加速。这种收益来自我们的程序感知调度器, 它能保持接近最优的 KV 缓存命中率 (Mini-SWE-Bench 和 OpenHands 上约为  $100\%$ , 见 Figure 5 a、b、d、e), 并支持环境异步准备。相比之下, Continuum 在高并发下会出现性能下降。如 Figure 5 所示, 它的 KV 缓存命中率从  $>90\%$  显著下降到  $\approx 60\%$ 。这是因为当没有足够显存用于正在进行的请求解码时, Continuum 会在不同程序请求之间发生 KV 缓存驱逐, 活动程序因争夺有限显存而引发系统抖动。

**高并发的鲁棒性能。** THUNDERAGENT 即使在并行工作流数量超出 GPU 显存限制时, 也能保持最大可实现吞吐量。如 Figure 4 所示, THUNDERAGENT 确保吞吐量随并行工作流数量增加仍保持稳定; 而

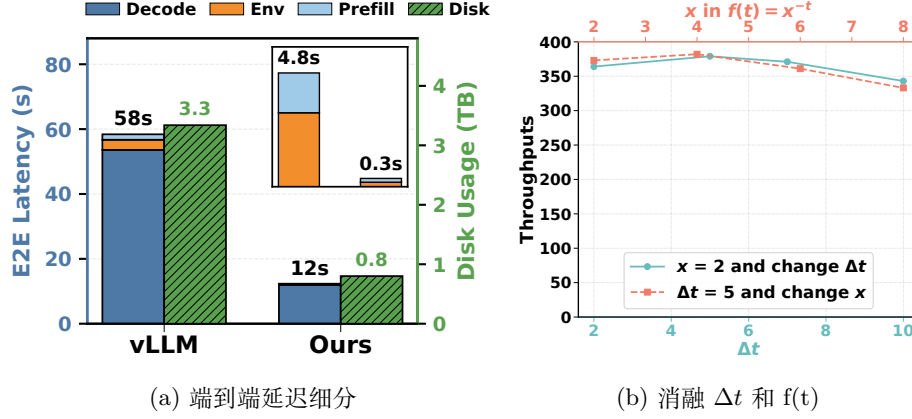


图 6: 端到端延迟分解和参数敏感性的消融研究 THUNDERAGENT.

一旦工作负载超过显存限制，基线系统就会出现严重吞吐崩溃。在实际智能体服务中，由于智能体环境和工具执行持续时间具有随机性，通常无法静态确定最佳并行工作流数量，以在有限 KV 缓存抖动和缓存成本下最大化利用率。THUNDERAGENT 通过自动适应最大可用容量而无需手动调参来解决此问题，这是稳健实际部署的关键功能。

**确定性和随机工具执行的鲁棒性。** THUNDERAGENT 不仅在具有确定性工具模式的工作流中优于基线 (Figure 4 a、b、d、e)，而且也在高度随机的条件下 (Figure 4 c、f)。这来自我们的动态程序感知等待队列策略。vLLM 的请求感知调度器通常不会为处于执行阶段的程序保留显存，从而迫使系统频繁重新计算。相反，Continuum 会为所有暂停程序静态保留显存，并依赖工具执行时间预测；当工具调用很长且不可预测时，这会带来高昂的  $\text{Cost}_{\text{recompute}}$  或  $\text{Cost}_{\text{caching}}$ 。THUNDERAGENT 通过时间衰减函数平衡它们  $f(t)$ ，优先为工具调用时间短的程序保留 KV 缓存，同时抢先暂停工具执行时间长的程序，以防止内存浪费。如图所示 Figure 5 (右)，虽然 THUNDERAGENT 在随机设置下，KV 缓存命中率低于 Continuum，它通过确保活动 GPU 利用率来实现更高的吞吐量。

### 5.3 Rollout 评估结果

我们在两节点 H100 集群上使用 GLM-4.6 评估 RL rollout (持续时间 3 小时)。Table 2 表明，THUNDERAGENT 保持了有效可扩展性，相比 vLLM + Gateway 基线实现 **1.79–3.92×** 吞吐提升，说明它非常适合显存密集型分布式强化学习工作负载。

表 2: 2×H100 节点上的 THUNDERAGENT GLM-4.6 rollout ( $N = 144$ )。

| Workflow      | Serving System | Throughput             |
|---------------|----------------|------------------------|
| mini-SWEAgent | vLLM + Gateway | 375.4                  |
| mini-SWEAgent | THUNDERAGENT   | 671.8 ( <b>1.79×</b> ) |
| OpenHands     | vLLM + Gateway | 69.1                   |
| OpenHands     | THUNDERAGENT   | 270.8 ( <b>3.92×</b> ) |

### 5.4 消融研究

**端到端延迟细分。** Figure 6a 分解了 OpenHands rollout 的平均端到端延迟。吞吐量提升主要来自 **预填充和解码延迟** 的降低。此外，工具资源管理策略 (节 4.4) 贡献了约 **10%** 的延迟改善，并带来 **4.2×**

的磁盘空间节省。每步端到端延迟见 附录 F.

**消融开启  $\Delta t$  和  $f(t)$ 。** 我们研究检测周期的灵敏度  $\Delta t$  和衰减功能  $f(t) = x^{-t}$ . Figure 6b 节目 THUNDERAGENT 在单个 H100 节点上以 GLM-4.6 作为基本模型离线服务 mini-SWEAgent。我们观察到 THUNDERAGENT 在不同参数设置下保持高吞吐量，证明了我们方法的稳健性。进一步增加  $\Delta t$  可能会降低 KV 缓存命中率，从而降低吞吐量，因为检测过程中可能会发生抖动。另外，增加  $x$  在  $f(t)$  允许更积极地驱逐执行程序，这些程序通过重新计算成本来降低缓存成本。这会降低智能体程序的吞吐量工具执行时间短会被过早驱逐。

## 6 结论

我们提出了 THUNDERAGENT，这是一个快速而简单的智能体系统，建立在程序级抽象之上，能够在每个智能体工作流的完整生命周期内跟踪元数据。THUNDERAGENT 利用这种程序抽象进行运行时调度和资源管理。具体而言，THUNDERAGENT 在 GPU 节点之间动态调度程序执行，以缓解 KV 缓存抖动和显存不均衡，同时管理工具资源以防止资源泄漏。实验结果表明，在服务场景中 THUNDERAGENT 相比已有系统提升 **1.48–3.58 $\times$** ，在 RL rollout 场景中提升 **1.79–3.92 $\times$** 。

## 7 致谢

我们感谢 Together.ai 使这项工作成为可能。感谢 Ben Athiwaratkun 和 Ce Zhang 在开发多后端调度器方面提供的帮助。感谢 Wenyi Hong 和 Luke Huang 在本工作期间提供的有益反馈和讨论。

## 参考文献

- [1] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966. doi: 10.1147/sj.52.0078.
- [2] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, Lawrence Jang, and Zack Hui. Windows agent arena: Evaluating multi-modal os agents at scale, 2024. URL <https://arxiv.org/abs/2409.08264>.
- [3] Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery, 2024. URL <https://arxiv.org/abs/2410.05080>.
- [4] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL <https://arxiv.org/abs/2505.24298>.



- [5] Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong, Weixun Wang, Dakai An, Lunxi Cao, Dilxat Muhtar, Zichen Liu, Haizhou Zhao, et al. Rollart: Scaling agentic rl training via disaggregated infrastructure. *arXiv preprint arXiv:2512.22560*, 2025.
- [6] Yiyuan He, Minxian Xu, Jingfeng Wu, Jianmin Hu, Chong Ma, Min Shen, Le Chen, Chengzhong Xu, Lin Qu, and Kejiang Ye. Banaserve: Unified kv cache and dynamic module migration for balancing disaggregated llm serving in ai infrastructure, 2025. URL <https://arxiv.org/abs/2510.13223>.
- [7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- [8] Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. Verltool: Towards holistic agentic reinforcement learning with tool use. *arXiv preprint arXiv:2509.01055*, 2025.
- [9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- [10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [11] Hanchen Li, Qiuyang Mang, Runyuan He, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live, 2025. URL <https://arxiv.org/abs/2511.02230>.
- [12] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. Lmcache: An efficient kv cache layer for enterprise-scale llm inference, 2025. URL <https://arxiv.org/abs/2510.09665>.
- [13] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An efficient serving engine for llm agents as general programs, 2025. URL <https://arxiv.org/abs/2502.13965>.
- [14] Emanuel Parzen. *Stochastic processes*. SIAM, 1999.
- [15] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, Michael Choi, Anish Agrawal, Arnav Chopra, Adam Khoja, Ryan Kim, Richard Ren, Jason Hausenloy, Oliver Zhang, Mantas Mazeika, Dmitry Dodonov, Tung Nguyen, Jaeho Lee, Daron Anderson, Mikhail Doroshenko, Alun Cennyth Stokes, Mobeen Mahmood, Oleksandr Pokutnyi, Oleg Iskra, Jessica P. Wang, John-Clark Levin, Mstyslav Kazakov, Fiona Feng, Steven Y. Feng, Haoran Zhao, Michael Yu, Varun Gangal, Chelsea Zou,

Zihan Wang, Serguei Popov, Robert Gerbicz, Geoff Galgon, Johannes Schmitt, Will Yeadon, Yongki Lee, Scott Sauers, Alvaro Sanchez, Fabian Giska, Marc Roth, Søren Riis, Saiteja Utpala, Noah Burns, Gashaw M. Goshu, Mohinder Maheshbhai Naiya, Chidozie Agu, Zachary Giboney, Antrell Cheatom, Francesco Fournier-Facio, Sarah-Jane Crowson, Lennart Finke, Zerui Cheng, Jennifer Zampese, Ryan G. Hoerr, Mark Nandor, Hyunwoo Park, Tim Gehringer, Jiaqi Cai, Ben McCarty, Alexis C Garretson, Edwin Taylor, Damien Sileo, Qiuyu Ren, Usman Qazi, Lianghui Li, Jungbae Nam, John B. Wydallis, Pavel Arkhipov, Jack Wei Lun Shi, Aras Bacho, Chris G. Willcocks, Hangrui Cao, Sumeet Motwani, Emily de Oliveira Santos, Johannes Veith, Edward Vendrow, Doru Cojoc, Kengo Zenitani, Joshua Robinson, Longke Tang, Yuqi Li, Joshua Vendrow, Natanael Wildner Fraga, Vladyslav Kuchkin, Andrey Pupasov Maksimov, Pierre Marion, Denis Efremov, Jayson Lynch, Kaiqu Liang, Aleksandar Mikov, Andrew Gritsevskiy, Julien Guillod, Gözdenur Demir, Dakotah Martinez, Ben Pageler, Kevin Zhou, Saeed Soori, Ori Press, Henry Tang, Paolo Rissone, Sean R. Green, Lina Brüssel, Moon Twayana, Aymeric Dieuleveut, Joseph Marvin Imperial, Ameya Prabhu, Jinzhou Yang, Nick Crispino, Arun Rao, Dimitri Zvonkine, Gabriel Loiseau, Mikhail Kalinin, Marco Lukas, Ciprian Manolescu, Nate Stambaugh, Subrata Mishra, Tad Hogg, Carlo Bosio, Brian P Coppola, Julian Salazar, Jaehyeok Jin, Rafael Sayous, Stefan Ivanov, Philippe Schwaller, Shaipranesh Senthilkuma, Andres M Bran, Andres Algaba, Kelsey Van den Houte, Lynn Van Der Sypt, Brecht Verbeken, David Noever, Alexei Kopylov, Benjamin Myklebust, Bikun Li, Lisa Schut, Evgenii Zheltonozhskii, Qiaochu Yuan, Derek Lim, Richard Stanley, Tong Yang, John Maar, Julian Wykowski, Martí Oller, Anmol Sahu, Cesare Giulio Ardito, Yuzheng Hu, Ariel Ghislain Kemogne Kamdoun, Alvin Jin, Tobias Garcia Vilchis, Yuexuan Zu, Martin Lackner, James Koppel, Gongbo Sun, Daniil S. Antonenko, Steffi Chern, Bingchen Zhao, Pierrot Arsene, Joseph M Cavanagh, Daofeng Li, Jiawei Shen, Donato Crisostomi, Wenjin Zhang, Ali Dehghan, Sergey Ivanov, David Perrella, Nurdin Kaparov, Allen Zang, Ilia Sucholutsky, Arina Kharlamova, Daniil Orel, Vladislav Poritski, Shalev Ben-David, Zachary Berger, Parker Whitfill, Michael Foster, Daniel Munro, Linh Ho, Shankar Sivarajan, Dan Bar Hava, Aleksey Kuchkin, David Holmes, Alexandra Rodriguez-Romero, Frank Sommerhage, Anji Zhang, Richard Moat, Keith Schneider, Zakayo Kazibwe, Don Clarke, Dae Hyun Kim, Felipe Meneguitti Dias, Sara Fish, Veit Elser, Tobias Kreiman, Victor Efren Guadarrama Vilchis, Immo Klose, Ujjwala Anantheswaran, Adam Zweiger, Kaivalya Rawal, Jeffery Li, Jeremy Nguyen, Nicolas Daans, Haline Heidinger, Maksim Radionov, Václav Rozhoň, Vincent Ginis, Christian Stump, Niv Cohen, Rafał Poświata, Josef Tkadlec, Alan Goldfarb, Chenguang Wang, Piotr Padlewski, Stanislaw Barzowski, Kyle Montgomery, Ryan Stendall, Jamie Tucker-Foltz, Jack Stade, T. Ryan Rogers, Tom Goertzen, Declan Grabb, Abhishek Shukla, Alan Givré, John Arnold Ambay, Archan Sen, Muhammad Fayez Aziz, Mark H Inlow, Hao He, Ling Zhang, Younesse Kaddar, Ivar Ångquist, Yanxu Chen, Harrison K Wang, Kalyan Ramakrishnan, Elliott Thornley, Antonio Terpin, Hailey Schoelkopf, Eric Zheng, Avishy Carmi, Ethan D. L. Brown, Kelin Zhu, Max Bartolo, Richard Wheeler, Martin Stehberger, Peter Bradshaw, JP Heimonen, Kaustubh Sridhar, Ido Akov, Jennifer Sandlin, Yury Makarychev, Joanna Tam, Hieu Hoang, David M. Cunningham, Vladimir Goryachev, Demosthenes Patrama-

nis, Michael Krause, Andrew Redenti, David Aldous, Jesyin Lai, Shannon Coleman, Jiangnan Xu, Sangwon Lee, Ilias Magoulas, Sandy Zhao, Ning Tang, Michael K. Cohen, Orr Paradise, Jan Hendrik Kirchner, Maksym Ovchynnikov, Jason O. Matos, Adithya Shenoy, Michael Wang, Yuzhou Nie, Anna Sztzyber-Betley, Paolo Faraboschi, Robin Riblet, Jonathan Crozier, Shiv Halasyamani, Shreyas Verma, Prashant Joshi, Eli Meril, Ziqiao Ma, Jérémy Andréoletti, Raghav Singhal, Jacob Platnick, Volodymyr Nevirkovets, Luke Basler, Alexander Ivanov, Seri Khoury, Nils Gustafsson, Marco Piccardo, Hamid Mostaghimi, Qijia Chen, Virendra Singh, Tran Quoc Khánh, Paul Rosu, Hannah Szlyk, Zachary Brown, Himanshu Narayan, Aline Menezes, Jonathan Roberts, William Alley, Kunyang Sun, Arkil Patel, Max Lamparth, Anka Reuel, Linwei Xin, Hanmeng Xu, Jacob Loader, Freddie Martin, Zixuan Wang, Andrea Achilleos, Thomas Preu, Tomek Korbak, Ida Bosio, Fereshteh Kazemi, Ziye Chen, Biró Bálint, Eve J. Y. Lo, Jiaqi Wang, Maria Inês S. Nunes, Jeremiah Milbauer, M Saiful Bari, Zihao Wang, Behzad Ansarinejad, Yewen Sun, Stephane Durand, Hossam Elgnainy, Guillaume Douville, Daniel Tordera, George Balabanian, Hew Wolff, Lynna Kvistad, Hsiaoyun Milliron, Ahmad Sakor, Murat Eron, Andrew Favre D. O., Shailesh Shah, Xiaoxiang Zhou, Firuz Kamalov, Sherwin Abdoli, Tim Santens, Shaul Barkan, Allison Tee, Robin Zhang, Alessandro Tomasiello, G. Bruno De Luca, Shi-Zhuo Looi, Vinh-Kha Le, Noam Kolt, Jiayi Pan, Emma Rodman, Jacob Drori, Carl J Fossum, Niklas Muennighoff, Milind Jagota, Ronak Pradeep, Honglu Fan, Jonathan Eicher, Michael Chen, Kushal Thaman, William Merrill, Moritz Firsching, Carter Harris, Stefan Ciobăcă, Jason Gross, Rohan Pandey, Ilya Gusev, Adam Jones, Shashank Agnihotri, Pavel Zhelnov, Mohammadreza Mofayezi, Alexander Piperski, David K. Zhang, Kostiantyn Dobarskyi, Roman Leventov, Ignat Soroko, Joshua Duersch, Vage Taamazyan, Andrew Ho, Wenjie Ma, William Held, Ruicheng Xian, Armel Randy Zebaze, Mohanad Mohamed, Julian Noah Leser, Michelle X Yuan, Laila Yacar, Johannes Lengler, Katarzyna Olszewska, Claudio Di Fratta, Edson Oliveira, Joseph W. Jackson, Andy Zou, Muthu Chidambaram, Timothy Manik, Hector Haffenden, Dashiell Stander, Ali Dasouqi, Alexander Shen, Bitu Golshani, David Stap, Egor Kreto, Mikalai Uzhou, Alina Borisovna Zhidkovskaya, Nick Winter, Miguel Orbegoza Rodriguez, Robert Lauff, Dustin Wehr, Colin Tang, Zaki Hosain, Shaun Phillips, Fortuna Samuele, Fredrik Ekström, Angela Hammon, Oam Patel, Faraz Farhidi, George Medley, Forough Mohammadzadeh, Madellene Peñaflor, Haile Kassahun, Alena Friedrich, Rayner Hernandez Perez, Daniel Pyda, Taom Sakal, Omkar Dhamane, Ali Khajegili Mirabadi, Eric Hallman, Kenchi Okutsu, Mike Battaglia, Mohammad Maghsoudimehrabani, Alon Amit, Dave Hulbert, Roberto Pereira, Simon Weber, Handoko, Anton Peristyy, Stephen Malina, Mustafa Mehkary, Rami Aly, Frank Reidegeld, Anna-Katharina Dick, Cary Friday, Mukhwinder Singh, Hassan Shapourian, Wanyoung Kim, Mariana Costa, Hubeyb Gurdogan, Harsh Kumar, Chiara Ceconello, Chao Zhuang, Haon Park, Micah Carroll, Andrew R. Tawfeek, Stefan Steinerberger, Daattavya Aggarwal, Michael Kirchhof, Linjie Dai, Evan Kim, Johan Ferret, Jainam Shah, Yuzhou Wang, Minghao Yan, Krzysztof Burdzy, Lixin Zhang, Antonio Franca, Diana T. Pham, Kang Yong Loh, Joshua Robinson, Abram Jackson, Paolo Giordano, Philipp Petersen, Adrian Cosma, Jesus Colino, Colin White, Jacob Votava, Vladimir Vinnikov, Ethan Delaney, Petr Spelda,

Vit Stritecky, Syed M. Shahid, Jean-Christophe Mourrat, Lavr Vetoshkin, Koen Sponselee, Renas Bacho, Zheng-Xin Yong, Florencia de la Rosa, Nathan Cho, Xiuyu Li, Guillaume Malod, Orion Weller, Guglielmo Albani, Leon Lang, Julien Laurendeau, Dmitry Kazakov, Fatimah Adesanya, Julien Portier, Lawrence Hollom, Victor Souza, Yuchen Anna Zhou, Julien Degorre, Yiğit Yalın, Gbenga Daniel Obikoya, Rai, Filippo Bigi, M. C. Boscá, Oleg Shumar, Kaniuar Bacho, Gabriel Recchia, Mara Popescu, Nikita Shulga, Ngefor Mildred Tanwie, Thomas C. H. Lux, Ben Rank, Colin Ni, Matthew Brooks, Alesia Yakimchyk, Huanxu, Liu, Stefano Cavalleri, Olle Häggström, Emil Verkama, Joshua Newbould, Hans Gundlach, Leonor Brito-Santana, Brian Amaro, Vivek Vajipey, Rynaa Grover, Ting Wang, Yosi Kratish, Wen-Ding Li, Sivakanth Gopi, Andrea Caciolai, Christian Schroeder de Witt, Pablo Hernández-Cámara, Emanuele Rodolà, Jules Robins, Dominic Williamson, Vincent Cheng, Brad Raynor, Hao Qi, Ben Segev, Jingxuan Fan, Sarah Martinson, Erik Y. Wang, Kaylie Hausknecht, Michael P. Brenner, Mao Mao, Christoph Demian, Peyman Kassani, Xinyu Zhang, David Avagian, Eshawn Jessica Scipio, Alon Ragoler, Justin Tan, Blake Sims, Rebeka Plecnik, Aaron Kirtland, Omer Faruk Bodur, D. P. Shinde, Yan Carlos Leyva Labrador, Zahra Adoul, Mohamed Zekry, Ali Karakoc, Tania C. B. Santos, Samir Shamseldeen, Loukmane Karim, Anna Liakhovitskaia, Nate Resman, Nicholas Farina, Juan Carlos Gonzalez, Gabe Maayan, Earth Anderson, Rodrigo De Oliveira Pena, Elizabeth Kelley, Hodjat Mariji, Rasoul Pouriamanesh, Wentao Wu, Ross Finocchio, Ismail Alarab, Joshua Cole, Danyelle Ferreira, Bryan Johnson, Mohammad Safdari, Liangti Dai, Siriphan Arthornthurasuk, Isaac C. McAlister, Alejandro José Moyano, Alexey Pronin, Jing Fan, Angel Ramirez-Trinidad, Yana Malysheva, Daphiny Pottmaier, Omid Taheri, Stanley Stepanic, Samuel Perry, Luke Askew, Raúl Adrián Huerta Rodríguez, Ali M. R. Minissi, Ricardo Lorena, Krishnamurthy Iyer, Arshad Anil Fasiludeen, Ronald Clark, Josh Ducey, Matheus Piza, Maja Somrak, Eric Vergo, Juehang Qin, Benjámín Borbás, Eric Chu, Jack Lindsey, Antoine Jallon, I. M. J. McInnis, Evan Chen, Avi Semler, Luk Gloor, Tej Shah, Marc Carauleanu, Pascal Lauer, Tran Duc Huy, Hossein Shahrtash, Emilien Duc, Lukas Lewark, Assaf Brown, Samuel Albanie, Brian Weber, Warren S. Vaz, Pierre Clavier, Yiyang Fan, Gabriel Poesia Reis e Silva, Long, Lian, Marcus Abramovitch, Xi Jiang, Sandra Mendoza, Murat Islam, Juan Gonzalez, Vasilios Mavroudis, Justin Xu, Pawan Kumar, Laxman Prasad Goswami, Daniel Bugas, Nasser Heydari, Ferenc Jeanplong, Thorben Jansen, Antonella Pinto, Archimedes Apronti, Abdallah Galal, Ng Ze-An, Ankit Singh, Tong Jiang, Joan of Arc Xavier, Kanu Priya Agarwal, Mohammed Berkani, Gang Zhang, Zhehang Du, Benedito Alves de Oliveira Junior, Dmitry Malishev, Nicolas Remy, Taylor D. Hartman, Tim Tarver, Stephen Mensah, Gautier Abou Loume, Wiktor Morak, Farzad Habibi, Sarah Hoback, Will Cai, Javier Gimenez, Roselynn Grace Montecillo, Jakub Łucki, Russell Campbell, Asankhaya Sharma, Khalida Meer, Shreen Gul, Daniel Espinosa Gonzalez, Xavier Alapont, Alex Hoover, Gunjan Chhablani, Freddie Vargus, Arunim Agarwal, Yibo Jiang, Deepakkumar Patil, David Outevsky, Kevin Joseph Scaria, Rajat Maheshwari, Abdelkader Dendane, Priti Shukla, Ashley Cartwright, Sergei Bogdanov, Niels Mündler, Sören Möller, Luca Arnaboldi, Kunvar Thaman, Muhammad Rehan Siddiqi, Prajvi Saxena, Himanshu Gupta, Tony Fruhauff, Glen Sherman, Mátyás Vincze, Siranut Usawasutsakorn, Dylan Ler, Anil

Radhakrishnan, Innocent Enyekwe, Sk Md Salauddin, Jiang Muzhen, Aleksandr Maksapetyan, Vivien Rossbach, Chris Harjadi, Mohsen Bahaloohoreh, Claire Sparrow, Jasdeep Sidhu, Sam Ali, Song Bian, John Lai, Eric Singer, Justine Leon Uro, Greg Bateman, Mohamed Sayed, Ahmed Menshawy, Darling Duclosel, Dario Bezzi, Yashaswini Jain, Ashley Aaron, Murat Tiryakioglu, Sheeshram Siddh, Keith Krenek, Imad Ali Shah, Jun Jin, Scott Creighton, Denis Peskoff, Zienab EL-Wasif, Ragavendran P V, Michael Richmond, Joseph McGowan, Tejal Patwardhan, Hao-Yu Sun, Ting Sun, Nikola Zubić, Samuele Sala, Stephen Ebert, Jean Kaddour, Manuel Schottdorf, Dianzhuo Wang, Gerol Petruzella, Alex Meiburg, Tilen Medved, Ali ElSheikh, S Ashwin Hebbbar, Lorenzo Vaquero, Xianjun Yang, Jason Poulos, Vilém Zouhar, Sergey Bogdanik, Mingfang Zhang, Jorge Sanz-Ros, David Anugraha, Yinwei Dai, Anh N. Nhu, Xue Wang, Ali Anil Demircali, Zhibai Jia, Yuyin Zhou, Juncheng Wu, Mike He, Nitin Chandok, Aarush Sinha, Gaoxiang Luo, Long Le, Mickaël Noyé, Michał Perelkiewicz, Ioannis Pantidis, Tianbo Qi, Soham Sachin Purohit, Letitia Parcalabescu, Thai-Hoa Nguyen, Genta Indra Winata, Edoardo M. Ponti, Hanchen Li, Kaustubh Dhole, Jongee Park, Dario Abbondanza, Yuanli Wang, Anupam Nayak, Diogo M. Caetano, Antonio A. W. L. Wong, Maria del Rio-Chanona, Dániel Kondor, Pieter Francois, Ed Chastrey, Jakob Zsambok, Dan Hoyer, Jenny Reddish, Jakob Hauser, Francisco-Javier Rodrigo-Ginés, Suchandra Datta, Maxwell Shepherd, Thom Kamphuis, Qizheng Zhang, Hyunjun Kim, Ruiji Sun, Jianzhu Yao, Franck Dernoncourt, Satyapriya Krishna, Sina Rismanchian, Bonan Pu, Francesco Pinto, Yingheng Wang, Kumar Shridhar, Kalon J. Overholt, Glib Briia, Hieu Nguyen, David, Soler Bartomeu, Tony CY Pang, Adam Wecker, Yifan Xiong, Fanfei Li, Lukas S. Huber, Joshua Jaeger, Romano De Maddalena, Xing Han Lü, Yuhui Zhang, Claas Beger, Patrick Tser Jern Kon, Sean Li, Vivek Sanker, Ming Yin, Yihao Liang, Xinlu Zhang, Ankit Agrawal, Li S. Yifei, Zechen Zhang, Mu Cai, Yasin Sonmez, Costin Cozianu, Changhao Li, Alex Slen, Shoubin Yu, Hyun Kyu Park, Gabriele Sarti, Marcin Briański, Alessandro Stolfo, Truong An Nguyen, Mike Zhang, Yotam Perlitz, Jose Hernandez-Orallo, Runjia Li, Amin Shabani, Felix Juefei-Xu, Shikhar Dhingra, Orr Zohar, My Chiffon Nguyen, Alexander Pondaven, Abdurrahim Yilmaz, Xuandong Zhao, Chuanyang Jin, Muyan Jiang, Stefan Todoran, Xinyao Han, Jules Kreuer, Brian Rabern, Anna Plassart, Martino Maggetti, Luther Yap, Robert Geirhos, Jonathon Kean, Dingsu Wang, Sina Mollaei, Chenkai Sun, Yifan Yin, Shiqi Wang, Rui Li, Yaowen Chang, Anjiang Wei, Alice Bizeul, Xiaohan Wang, Alexandre Oliveira Arrais, Kushin Mukherjee, Jorge Chamorro-Padial, Jiachen Liu, Xingyu Qu, Junyi Guan, Adam Bouyamourn, Shuyu Wu, Martyna Plomecka, Junda Chen, Mengze Tang, Jiaqi Deng, Shreyas Subramanian, Haocheng Xi, Haoxuan Chen, Weizhi Zhang, Yinuo Ren, Haoqin Tu, Sejong Kim, Yushun Chen, Sara Vera Marjanović, Junwoo Ha, Grzegorz Luczyna, Jeff J. Ma, Zewen Shen, Dawn Song, Cedegao E. Zhang, Zhun Wang, Gaël Gendron, Yunze Xiao, Leo Smucker, Erica Weng, Kwok Hao Lee, Zhe Ye, Stefano Ermon, Ignacio D. Lopez-Miguel, Theo Knights, Anthony Gitter, Namkyu Park, Boyi Wei, Hongzheng Chen, Kunal Pai, Ahmed Elkhany, Han Lin, Philipp D. Siedler, Jichao Fang, Ritwik Mishra, Károly Zsolnai-Fehér, Xilin Jiang, Shadab Khan, Jun Yuan, Rishab Kumar Jain, Xi Lin, Mike Peterson, Zhe Wang, Aditya Malusare, Maosen Tang, Isha Gupta, Ivan Fosin, Timothy Kang, Barbara



Dworakowska, Kazuki Matsumoto, Guangyao Zheng, Gerben Sewuster, Jorge Pretel Villanueva, Ivan Rannev, Igor Chernyavsky, Jiale Chen, Deepayan Banik, Ben Racz, Wenchao Dong, Jianxin Wang, Laila Bashmal, Duarte V. Gonçalves, Wei Hu, Kaushik Bar, Ondrej Bohdal, Atharv Singh Patlan, Shehzaad Dhuliawala, Caroline Geirhos, Julien Wist, Yuval Kansal, Bingsen Chen, Kutay Tire, Atak Talay Yücel, Brandon Christof, Veerupaksh Singla, Zijian Song, Sanxing Chen, Jiaxin Ge, Kaustubh Ponkshe, Isaac Park, Tianneng Shi, Martin Q. Ma, Joshua Mak, Sherwin Lai, Antoine Moulin, Zhuo Cheng, Zhanda Zhu, Ziyi Zhang, Vaidehi Patil, Ketan Jha, Qiutong Men, Jiaxuan Wu, Tianchi Zhang, Bruno Hebling Vieira, Alham Fikri Aji, Jae-Won Chung, Mohammed Mahfoud, Ha Thi Hoang, Marc Sperzel, Wei Hao, Kristof Meding, Sihan Xu, Vassilis Kostakos, Davide Manini, Yueying Liu, Christopher Toukmaji, Jay Paek, Eunmi Yu, Arif Engin Demircali, Zhiyi Sun, Ivan Dewerpe, Hongsen Qin, Roman Pflugfelder, James Bailey, Johnathan Morris, Ville Heilala, Sybille Rosset, Zishun Yu, Peter E. Chen, Woongyeong Yeo, Eeshaan Jain, Ryan Yang, Sreekar Chigurupati, Julia Chernyavsky, Sai Prajwal Reddy, Subhashini Venugopalan, Hunar Batra, Core Francisco Park, Hieu Tran, Guilherme Maximiano, Genghan Zhang, Yizhuo Liang, Hu Shiyu, Rongwu Xu, Rui Pan, Siddharth Suresh, Ziqi Liu, Samaksh Gulati, Songyang Zhang, Peter Turchin, Christopher W. Bartlett, Christopher R. Scotese, Phuong M. Cao, Ben Wu, Jacek Karwowski, Davide Scaramuzza, Aakaash Nattanmai, Gordon McKellips, Anish Cheraku, Asim Suhail, Ethan Luo, Marvin Deng, Jason Luo, Ashley Zhang, Kavin Jindel, Jay Paek, Kasper Halevy, Allen Baranov, Michael Liu, Advait Avadhanam, David Zhang, Vincent Cheng, Brad Ma, Evan Fu, Liam Do, Joshua Lass, Hubert Yang, Surya Sunkari, Vishruth Bharath, Violet Ai, James Leung, Rishit Agrawal, Alan Zhou, Kevin Chen, Tejas Kalpathi, Ziqi Xu, Gavin Wang, Tyler Xiao, Erik Maung, Sam Lee, Ryan Yang, Roy Yue, Ben Zhao, Julia Yoon, Sunny Sun, Aryan Singh, Ethan Luo, Clark Peng, Tyler Osbey, Taozhi Wang, Daryl Echeazu, Hubert Yang, Timothy Wu, Spandan Patel, Vidhi Kulkarni, Vijaykaarti Sundarapandian, Ashley Zhang, Andrew Le, Zafir Nasim, Srikar Yalam, Ritesh Kasamsetty, Soham Samal, Hubert Yang, David Sun, Nihar Shah, Abhijeet Saha, Alex Zhang, Leon Nguyen, Laasya Nagumalli, Kaixin Wang, Alan Zhou, Aidan Wu, Jason Luo, Anwith Telluri, Summer Yue, Alexandr Wang, and Dan Hendrycks. Humanity’s last exam, 2025. URL <https://arxiv.org/abs/2501.14249>.

- [16] Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL’s razor: Why online reinforcement learning forgets less, 2025. URL <https://arxiv.org/abs/2509.04259>.
- [17] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys ’25, pp. 1279–1297. ACM, March 2025. doi: 10.1145/3689031.3696075. URL <http://dx.doi.org/10.1145/3689031.3696075>.
- [18] Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, Yi Dong, Evelina Bakhturina, Tao Yu, Yejin Choi, Jan Kautz, and

- Pavlo Molchanov. Toolorchestra: Elevating intelligence via efficient model and tool orchestration, 2025. URL <https://arxiv.org/abs/2511.21689>.
- [19] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference, 2025. URL <https://arxiv.org/abs/2410.21465>.
- [20] 5 Team, Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, Kedong Wang, Lucen Zhong, Mingdao Liu, Rui Lu, Shulin Cao, Xiaohan Zhang, Xuancheng Huang, Yao Wei, Yean Cheng, Yifan An, Yilin Niu, Yuanhao Wen, Yushi Bai, Zhengxiao Du, Zihan Wang, Zilin Zhu, Bohan Zhang, Bosi Wen, Bowen Wu, Bowen Xu, Can Huang, Casey Zhao, Changpeng Cai, Chao Yu, Chen Li, Chendi Ge, Chenghua Huang, Chenhui Zhang, Chenxi Xu, Chenzheng Zhu, Chuang Li, Congfeng Yin, Daoyan Lin, Dayong Yang, Dazhi Jiang, Ding Ai, Erle Zhu, Fei Wang, Gengzheng Pan, Guo Wang, Hailong Sun, Haitao Li, Haiyang Li, Haiyi Hu, Hanyu Zhang, Hao Peng, Hao Tai, Haoke Zhang, Haoran Wang, Haoyu Yang, He Liu, He Zhao, Hongwei Liu, Hongxi Yan, Huan Liu, Huilong Chen, Ji Li, Jiajing Zhao, Jiamin Ren, Jian Jiao, Jiani Zhao, Jianyang Yan, Jiaqi Wang, Jiayi Gui, Jiayue Zhao, Jie Liu, Jijie Li, Jing Li, Jing Lu, Jingsen Wang, Jingwei Yuan, Jingxuan Li, Jingzhao Du, Jinhua Du, Jinxin Liu, Junkai Zhi, Junli Gao, Ke Wang, Lekang Yang, Liang Xu, Lin Fan, Lindong Wu, Lintao Ding, Lu Wang, Man Zhang, Minghao Li, Minghuan Xu, Mingming Zhao, Mingshu Zhai, Pengfan Du, Qian Dong, Shangde Lei, Shangqing Tu, Shangtong Yang, Shaoyou Lu, Shijie Li, Shuang Li, Shuang-Li, Shuxun Yang, Sibao Yi, Tianshu Yu, Wei Tian, Weihai Wang, Wenbo Yu, Weng Lam Tam, Wenjie Liang, Wentao Liu, Xiao Wang, Xiaohan Jia, Xiaotao Gu, Xiaoying Ling, Xin Wang, Xing Fan, Xingru Pan, Xinyuan Zhang, Xinze Zhang, Xiuqing Fu, Xunkai Zhang, Yabo Xu, Yandong Wu, Yida Lu, Yidong Wang, Yilin Zhou, Yiming Pan, Ying Zhang, Yingli Wang, Yingru Li, Yinpei Su, Yipeng Geng, Yitong Zhu, Yongkun Yang, Yuhang Li, Yuhao Wu, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yuxuan Zhang, Zezhen Liu, Zhen Yang, Zhengda Zhou, Zhongpei Qiao, Zhuoer Feng, Zhuorui Liu, Zichen Zhang, Zihan Wang, Zijun Yao, Zikang Wang, Ziqiang Liu, Ziwei Chai, Zixuan Li, Zuodong Zhao, Wenguang Chen, Jidong Zhai, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models, 2025. URL <https://arxiv.org/abs/2508.06471>.
- [21] vLLM Team. KV-aware Routing — vLLM Production Stack Documentation. <https://docs.vllm.ai/projects/production-stack/en/vllm-stack-0.1.5/tutorials/kvaware.html>, 2025. Accessed: 2026-01-15.
- [22] Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, et al. Let it flow: Agentic crafting on rock and roll, building the rome model within an open agentic learning ecosystem. *arXiv preprint arXiv:2512.24873*, 2025.
- [23] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng,

- Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents, 2025. URL <https://arxiv.org/abs/2407.16741>.
- [24] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
- [25] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. Strata: Hierarchical context caching for long context language model serving, 2025. URL <https://arxiv.org/abs/2508.18572>.
- [26] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [27] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [28] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys ’ 25, pp. 144–158. ACM, March 2025. doi: 10.1145/3689031.3696086. URL <http://dx.doi.org/10.1145/3689031.3696086>.
- [29] Yichao Yuan, Advait Iyer, Lin Ma, and Nishil Talati. Vortex: Overcoming memory capacity limitations in gpu-accelerated large-scale data analytics, 2025. URL <https://arxiv.org/abs/2502.09541>.
- [30] Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, et al. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework. *arXiv preprint arXiv:2510.04206*, 2025.
- [31] Lei Zhang, Mouxiang Chen, Ruisheng Cao, Jiawei Chen, Fan Zhou, Yiheng Xu, Jiaxi Yang, Liang Chen, Changwei Luo, Kai Zhang, et al. Megaflow: Large-scale distributed orchestration system for the agentic era. *arXiv preprint arXiv:2601.07526*, 2026.

- [32] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, and Junyang Lin. Stabilizing reinforcement learning with llms: Formulation and practices, 2025. URL <https://arxiv.org/abs/2512.01374>.
- [33] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024. URL <https://arxiv.org/abs/2312.07104>.
- [34] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024. URL <https://arxiv.org/abs/2401.09670>.
- [35] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024. URL <https://arxiv.org/abs/2307.13854>.

## A 与先前工作的扩展比较

### A.1 KV 缓存优化

**多层 KV 缓存管理。** 为了缓解 GPU 显存压力, Pensieve 等系统 [28], Continuum [11], Strata [25], 以及 ShadowKV [19] 利用硬件内存层次结构, 包括 GPU HBM、CPU DRAM 和 NVMe SSD 进行 KV 缓存管理。这些分层缓存机制通过将不活动的 KV 状态卸载到较低层存储并在请求恢复时将它们预取回 GPU 来减轻瞬时抢占。然而, 这些方法的实际效率从根本上受到设备和主机存储器之间的层间带宽的限制。在高频智能体 workflow 中, 频繁换入和换出周期的开销通常会抵消多层缓存的优势, 如下所示节 5.

**分布式 KV 缓存管理。** 智能体状态的分布式管理给 KV 缓存驱逐和抢占策略带来了极大的复杂性。而像 BanaServe 这样的系统 [6] 以及 LMCache [12] 虽然跨 DP 节点启用 KV 缓存传输, 但它们在大批量智能体服务和部署中的性能通常受到有限互连带宽的限制。智能体 workflow 中强大的程序内依赖性需要频繁的状态传输, 而无需程序级管理, 这很容易在服务或部署期间使网络饱和。

为了绕过这些带宽瓶颈, 标准推理系统如 vLLM KV 感知路由器 [21] 和 SGLang 模型网关 [33] 采用 KV 感知路由策略, 根据前缀位置或会话 ID 将请求固定到特定节点。同样, Vortex [29] 引入会话感知预取以最小化跨节点数据传输延迟。然而, 这些方法缺乏在 DP 节点之间动态迁移活动程序状态的能力。缺乏工作负载转移会导致整个集群内存利用率严重不平衡, 如图所示 Figure 2a。托管长时间运行的智能体程序的节点无法将状态卸载到空闲对等点, 从而导致资源利用率碎片化和总吞吐量下降。

### A.2 KV 缓存优化的扩展实验结果

**KV 缓存卸载实验。** 我们使用 LMCache 研究了 KV 缓存卸载 [12] 作为解决容量限制的潜在补救措施。虽然卸载理论上可以利用 CPU 或 SSD 存储来扩展有效内存空间, 但我们使用 vLLM + LMCache 的实现揭示了一个关键瓶颈: PCIe 带宽不足以维持智能体工作负载固有的高频上下文切换和大容量数据传输。如图所示 Figure 7a, 服务 GLM-4.6 型号时 [20] 配合 mini-SWEAgent 框架 [27], 频繁的换入和换出操作带来的延迟损失抵消了 GPU 显存容量的优势, 导致在繁重的智能体工作负载下吞吐量严重下降。

**预填充解码 (PD) 分解实验。** 我们还探索了 PD 分解 [34], 一种聊天机器人服务的标准优化, 通过将解码阶段与预填充干扰隔离开来。然而, 当应用于以持续上下文增长为特征的智能体工作负载时, 我们观察到 PD 分解 加剧抖动。通过将集群划分为仅预填充和仅解码节点, 可用于处理预填充的有效 HBM 池明显小于统一架构中的 HBM 池。这种内存碎片会导致系统达到容量限制并在低得多的并发级别下触发动抖, 如图所示 Figure 7b。这些结果表明, 通用架构优化不能替代主动管理工作集的以程序为中心的调度器。

### A.3 扩大智能体 workflow

**异构资源分配和调度。** 为了大规模协调多轮智能体与环境的交互, 最近的系统 (例如 MegaFlow ) [31], RollArt [5, 22], AgentRL [30], VerlTool [8] 将模型推理与环境执行分离。虽然这些框架通过专门的服务有效地扩展了环境并发性, 但它们表现出了粗粒度分解的固有局限性。这些系统将推理引擎和

工具执行器视为孤立的黑匣子，缺乏统一的资源管理，无法协调 KV 缓存生命周期与环境执行。如果没有程序级的细粒度调度，基于分解的方法会浪费智能体工作负载中的 KV 缓存重用潜力，从而产生次优吞吐量。



## B 系统可移植性和接口抽象。

### B.1 中间件架构和统一接口。

THUNDERAGENT 充当 **程序感知运行时层**通过程序级抽象在智能体控制流和后端推理引擎之间进行调解。调度器根据抽象的状态控制程序状态转换 `ProgramState`（见表 3a）连同后端缓存容量视图（见表 4）。同时，每个程序仅绑定到端点，不依赖于具体的后端实现。

### B.2 为什么程序 ID 很重要。

虽然标准 **会话 ID** 作为路由标签，**我们的系统使用程序 ID 来检查工作流元数据**。这种可见性至关重要：它允许调度器区分有效的工具等待时间和空闲会话，从而实现基于会话的基线无法支持的智能抢占策略。

### B.3 ThunderAgent 的低开销采用。

图 8 表明采用 ThunderAgent **只需要附着 program\_id 请求**（LLM 推理和工具执行）并发送明确的释放信号 `program_id` 当一个程序结束时。这 `program_id` 用自己的程序实例标记每个请求以进行调度，而释放信号允许 ThunderAgent 在终止后回收每个程序的资源。**所有其他请求字段和 OpenAI 风格的 API 界面保持不变。**

| 场地           | 类型   | 意义            | 地位        | 意义              |
|--------------|------|---------------|-----------|-----------------|
| 程序状态         |      |               | 节目状态      |                 |
| status       | 节目状态 | 当前生命周期状态。     | REASONING | GPU 上的推理。       |
| backend_url  | 斯特   | 分配的后端端点。      | ACTING    | Off-GPU 工具执行程序。 |
| step_count   | 整数   | 到目前为止已执行的步骤。  | PAUSED    | 在全局暂停等待集中。      |
| total_tokens | 整数   | 完整历史记录中的代币总数。 | STOPPED   | 已发布；资源被回收。      |

(a) 程序状态字段。

(b) 程序状态语义。

表 3: 程序状态和状态定义。

| 场地                    | 类型        | 意义              |
|-----------------------|-----------|-----------------|
| 后端状态                  |           |                 |
| url                   | 斯特        | 后端端点。           |
| healthy               | 布尔值       | 用于调度的健康标志。      |
| cache_config          | 可选 [缓存配置] | 静态缓存配置（在启动时获取）。 |
| active_program_tokens | 整数        | 此后端上的活动令牌足迹。    |

表 4: BackendState 的关键字段。

## C 工具执行时间可变性。

实际的智能体工具调用很难表征并且通常是不可预测的。在一些以代码为中心的设置中（例如，服务 SWE-Bench [9] 与 SWE-Agent [27] 或 OpenHands [23]），智能体主要调用本地的轻量级工具，工具延迟相对稳定，方差较小。然而，在更广泛和更现实的场景中，例如服务 HLE [15] 与 ToolOrchestra 一起 [18]，工作负载更加依赖远程服务工具（表 5），使得工具执行时间不稳定且难以预测。这种波动很大程度上源于智能体运行时的外部因素，例如网络抖动、后端负载和排队延迟以及速率限制，这些因素可能会因请求和时间而变化。

我们凭经验证实了图中的这种行为 9。对于远程服务工具（以及一些执行工具），中位数和尾分位数之间的差距很大：p95 和 p99 大大高于中位数，并且尾部可以延长到数十甚至数百秒。这表明这些设置中的工具延迟缺乏稳定的集中趋势；相反，重尾行为占主导地位，使得工具延迟预测在实践中本质上很脆弱。

考虑到工具执行的不可预测性，低估会浪费固定的缓存容量，同时仍然会触发过早的 KV 驱逐，导致恢复时出现抖动。相反，高估可能会导致不必要地驱逐本应保留的程序的 KV。即使工具运行时间是完全可预测的，现有的方法（例如 Continuum）[11] 仍然决定是否使用静态的、基于阈值的规则来保持 KV 缓存固定。相比之下，ThunderAgent 构建完整的成本建模框架和动态权衡  $\text{Cost}_{\text{recompute}}$  和  $\text{Cost}_{\text{caching}}$ 。

| 工具桶                      | 角色           | 主要变异源           |
|--------------------------|--------------|-----------------|
| HLE-search               | 检索证据         | 远程服务（网络延迟/速率限制） |
| HLE-enhance-reasoning    | 模型即工具调用      | 远程服务            |
| HLE-answer               | 最终一代         | 本地 LLM 推理       |
| SAB-execute_bash         | 外壳执行         | 沙箱和 I/O         |
| SAB-execute_ipython_cell | Python 单元格执行 | 程序运行时间          |
| SAB-str_replace_editor   | 文件编辑         | 本地文件系统          |
| SAB-task_tracker         | 任务状态跟踪       | 本地文件系统          |

表 5: 工具桶。

## D KV 缓存命中率统计及解读

在我们的成本分解方程中 (3)，智能体服务的吞吐量损失主要来自于 非生产性的间接费用：外部工具执行期间由抖动和空闲 KV 缓存引起的 KV 重新计算，即  $\text{Cost}_{\text{recompute}}$  和  $\text{Cost}_{\text{caching}}$ 。当工具调用较短且可预测时，作用阶段仅占用 KV 很短的时间，因此  $\text{Cost}_{\text{caching}}$  很小；因此，避免抖动占主导地位：较高的 KV 缓存命中率通常意味着更少的重新预填充和更高的吞吐量。

然而，当工具执行时间变化很大时（参见附录 C），基于 TTL 的调度器最终可能会为长工具调用固定 KV。虽然这可以减少  $\text{Cost}_{\text{recompute}}$  并提高 KV 缓存命中率，但同时会膨胀  $\text{Cost}_{\text{caching}}$  并降低吞吐量。这有助于解释为什么 Continuum [11] 尽管实现了更高的 KV 缓存命中率，却仍可能在工具繁重的工作负载上表现不佳（图 4, 5）。

**ThunderAgent 通过显式平衡缓存和重新计算来适应这些制度。** ThunderAgent 在第 4.3 节中为执行阶段程序引入时间衰减函数  $f(t)$ ，用于权衡  $\text{Cost}_{\text{caching}}$  和  $\text{Cost}_{\text{recompute}}$ ；我们在附录 E.1 中严格推导了  $f(t)$  的最优函数形式。通过逐步降低长时间空闲执行程序的有效显存优先级，调度器会驱逐它们的 KV 缓存，从而在控制重新计算的同时减少空闲缓存成本，并在实践中带来更高吞吐量（图 4）。

## E 扩展的理论分析。

### E.1 用于周期性抖动检测的时间衰减函数证明。

**Hypothesis E.1** (Unpredictable Tool Execution Time). 对于执行程序，我们假设调度器无法可靠地预测给定程序的工具返回时间（参见附录 C）。因此，衰减函数  $f$  应该只以时间均匀的方式依赖已经经过的执行时间  $t$  [14]。

**Hypothesis E.2** (Boundary Conditions). 我们假设时间衰减函数  $f : [0, \infty) \rightarrow (0, 1]$  满足

$$f(0) = 1, \lim_{t \rightarrow \infty} f(t) = 0, \quad (12)$$

这些边界条件的直观解释是，当工具执行时间为 0，对应于没有工具调用的多轮交互，所有的智能体程序都简化为推理程序，因此  $f(t) = 1$ 。相反，如果工具执行时间是无限的，智能体工作流就会崩溃为单轮生成，类似于标准聊天机器人服务，因为请求永远不会返回下一轮交互。在此制度下，设置  $f(t) = 0$  将衰减函数与请求级调度策略保持一致。

**Theorem E.1** (Admissible Time Decay Functions). 假设下 E.1 和 E.2，允许的时间衰减函数  $f$  方程中的容量检查函数 7 必须采用以下形式之一：连续时间的指数， $f(t) = e^{-\lambda t}$  和  $\lambda > 0$ ，或离散滴答时间的几何， $f(k) = x^{-k}$  和  $x > 1$ 。

证明。我们通过首先形式化隐含的时间齐次性质来证明这个定理假设 E.1。接下来，我们引入允许的时间衰减函数  $f$  在边界条件下假设 E.2。

**不可预测的工具时间的形式化。** 让  $t$  表示经过的作用时间，以挂钟时间（连续时间）或周期监视器滴答声（离散时间）测量。假设下 E.1，等待额外持续时间后的相对衰减  $\Delta$  不应取决于绝对经过的时间  $t$ ，但仅限于增量  $\Delta$ 。我们将其形式化为函数的存在  $\phi : [0, \infty) \rightarrow (0, 1]$  这样，对于所有  $t, \Delta \geq 0$ ,

$$f(t + \Delta) = f(t) \phi(\Delta). \quad (13)$$

**半群方程。** 环境  $t = 0$  在方程中 13 并使用边界条件  $f(0) = 1$ （来自假设 E.2）产量  $\phi(\Delta) = f(\Delta)$ 。代入回去，我们得到乘法半群方程

$$f(t + \Delta) = f(t) f(\Delta), \quad \forall t, \Delta \geq 0. \quad (14)$$

**连续时间情况（指数衰减）。** 我们首先考虑连续时间的情况。定义  $h(t) \triangleq \ln f(t)$ 。两边取对数方程 14 产生柯西函数方程

$$h(t + \Delta) = h(t) + h(\Delta). \quad (15)$$

自从  $f(t) \in (0, 1]$ ，我们有  $h(t) \leq 0$  为所有人  $t \geq 0$ ，这意味着那  $h$  上界为  $[0, \infty)$ 。在这种有界条件下，柯西函数方程只允许线性解形式的  $h(t) = ct$  对于某些人来说  $c \in \mathbb{R}$ 。写作  $\lambda \triangleq -c \geq 0$ ，我们得到

$$f(t) = e^{-\lambda t}. \quad (16)$$

最后，边界条件  $\lim_{t \rightarrow \infty} f(t) = 0$ （假设 E.2）排除  $\lambda = 0$ ，因此  $\lambda > 0$ 。

**离散时间情况（几何衰减）。** 接下来我们考虑离散时间设置，其中经过的作用时间是以整数刻度测量  $k \in \mathbb{Z}_{\geq 0}$ 。方程 14 变成

$$f(m + n) = f(m) f(n), \quad \forall m, n \in \mathbb{Z}_{\geq 0}. \quad (17)$$

环境  $n = 1$  产生递归  $f(k) = f(k-1)f(1)$ 。让  $\gamma \triangleq f(1)$ ，我们有  $f(k) = f(1)^k \triangleq \gamma^k$ 。边界条件  $\lim_{k \rightarrow \infty} f(k) = 0$  意味着  $0 < \gamma < 1$ 。等价地，我们可以参数化

$$f(k) = x^{-k}, \quad x \triangleq \gamma^{-1} > 1. \quad (18)$$

这样就完成了证明。  $\square$

## E.2 重新计算 STP 成本的证明

正如定义在 节 4.2，STP 重新计算成本由下式给出：

$$\text{Cost}_{\text{recompute}} = \int_0^{t_{\text{recompute}}} c_i(t) dt \quad (19)$$

在哪里  $c_i(t)$  表示瞬时成本，与解码步骤成正比（即  $c_i(t) \propto t$ ）。出现这种比例的原因是分块预填充每次迭代处理恒定数量的 KV 对，从而导致累积计算随着时间的推移线性增加。因此，评估积分收益率  $\text{Cost}_{\text{recompute}} \propto t_{\text{recompute}}^2$ 。鉴于关系  $t_{\text{recompute}} = c_i \times T_{\text{decode}}/\text{chunk}$ ，其中两者  $T_{\text{decode}}$  并且块大小是恒定的，因此：

$$\text{Cost}_{\text{recompute}} \propto c_i^2$$

## E.3 最小化重新计算 STP 成本的证明

我们使用以下方法为最短优先驱逐策略的最优性提供了严格的证明：**交换论点**。

**问题定义**。我们的目标是选择暂停程序的子集  $S$  驱逐使得总回收内存满足  $\sum_{i \in S} c_i \geq \Delta C$ ，同时最小化总重新计算成本  $J(S) = \sum_{i \in S} c_i^2$ 。注意成本函数  $f(x) = x^2$  是严格凸且超可加的（即  $(a+b)^2 > a^2 + b^2$  为积极的  $a, b$ ）。

**定理**。最小化的最优策略  $J(S)$  就是严格选择上下文长度最小的程序  $c_i$ 。

**证明**。为了矛盾起见，假设最优集  $S^*$  是不是最短的程序集。这意味着存在一个“长”程序  $p_{\text{long}} \in S^*$  和一个“短”节目  $p_{\text{short}} \notin S^*$ （可用但未选择）使得  $c_{\text{short}} < c_{\text{long}}$ 。

我们可以构造一个新集合  $S'$  通过交换或分解  $p_{\text{long}}$ 。自从  $c_{\text{long}} > c_{\text{short}}$ ，我们可以概念化  $p_{\text{long}}$  由一段长度组成  $c_{\text{short}}$  和残留物  $r = c_{\text{long}} - c_{\text{short}}$ 。

替换选择  $p_{\text{long}}$  和  $p_{\text{short}}$ （理论上剩余  $r$ ）改变成本。考虑由平方函数的凸性导出的不等式：

$$c_{\text{long}}^2 = (c_{\text{short}} + r)^2 = c_{\text{short}}^2 + r^2 + 2c_{\text{short}}r \quad (20)$$

自从  $c_{\text{short}} > 0$  和  $r > 0$ ，交叉项  $2c_{\text{short}}r > 0$ 。所以：

$$c_{\text{short}}^2 + r^2 < c_{\text{long}}^2 \quad (21)$$

这种不平等意味着打破一个大的驱逐目标 ( $c_{\text{long}}$ ) 分解为更小的组件 ( $c_{\text{short}} + r$ ) 严格减少平方和。在我们的调度器的上下文中，这意味着如果我们满足内存约束  $\Delta C$  使用大型程序，我们可以通过将其替换为总容量相同的可用较小程序（或其组合）来严格减少惩罚。

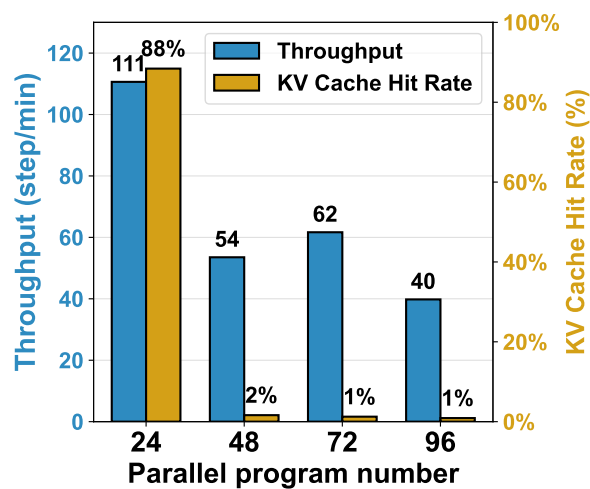
通过迭代地应用这种交换——用较小的未选择的程序替换最大的选定程序——我们单调地降低成本函数  $J(S)$ 。仅当不可能进行此类交换时，即当  $S$  完全由具有最小可用上下文长度的程序组成。

**结论**。最短优先策略是全局最优的，因为注意力的超线性成本 ( $O(L^2)$ ) 对碎片的惩罚小于对聚合的惩罚。

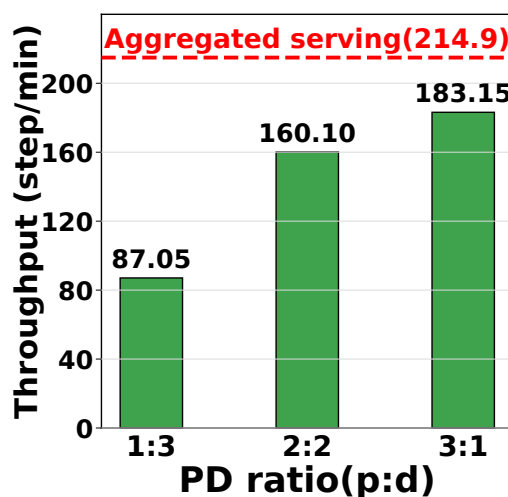
## F 端到端延迟分析

尽管我们已在 节 1 中说明, 对于自主智能体和智能体 RL rollout, 程序级延迟 (生成整个工作流所用时间) 比每步端到端延迟更重要, 这里仍比较 THUNDERAGENT、vLLM 和 Continuum 的平均每步延迟。Figure 10 表明, 在单个 H100 上以低或高并行工作流数量运行 GLM-4.6 和 Qwen3 235B, 并搭配 mini-SWEAgent 与 OpenHands 时, THUNDERAGENT 显著优于 vLLM 和 Continuum。**Continuum** 看似通过切换执行阶段程序改善了端到端延迟, 但实际上会触发严重 KV 缓存抖动, 从而推迟所有运行中程序的完成。





(a) LMCache 卸载的 KV 缓存命中率



(b) 吞吐量对比 Prefill-Only/Decode-Only Node Ratio

图 7: KV 缓存卸载和预填充解码 (PD) 分解的消融研究

#### 仅推理后端 (例如 vLLM/SGLang)

```

1 # 1) LLM request
2 chat_completion(model_id, messages, extrabody)
3
4 # 2) Tool execution
5 run_tool(command, sandbox)
6
7 # 3) Program end
8 # (no explicit release)

```

#### With ThunderAgent

```

1 # 1) LLM request
2 extrabody["program_id"] = "PID"
3 chat_completion(model_id, messages, extrabody)
4
5 # 2) Tool execution
6 run_tool(command, sandbox, program_id="PID")
7
8 # 3) Program end (explicit release)
9 POST /programs/release
10 { "program_id": "PID" }

```

图 8: 使用 ThunderAgent 只需要进行三处更改。

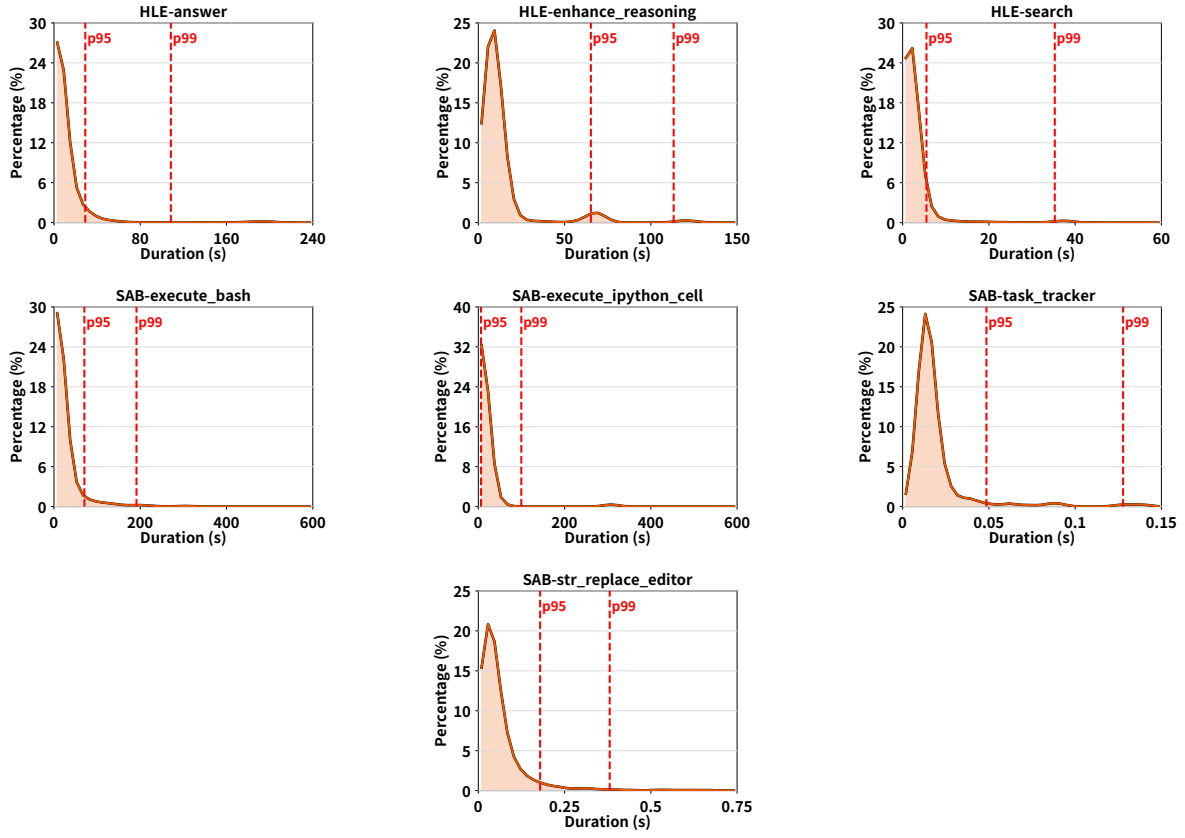


图 9: 工具执行时间分布。工具执行时间表现出高度可变性并且难以预测。

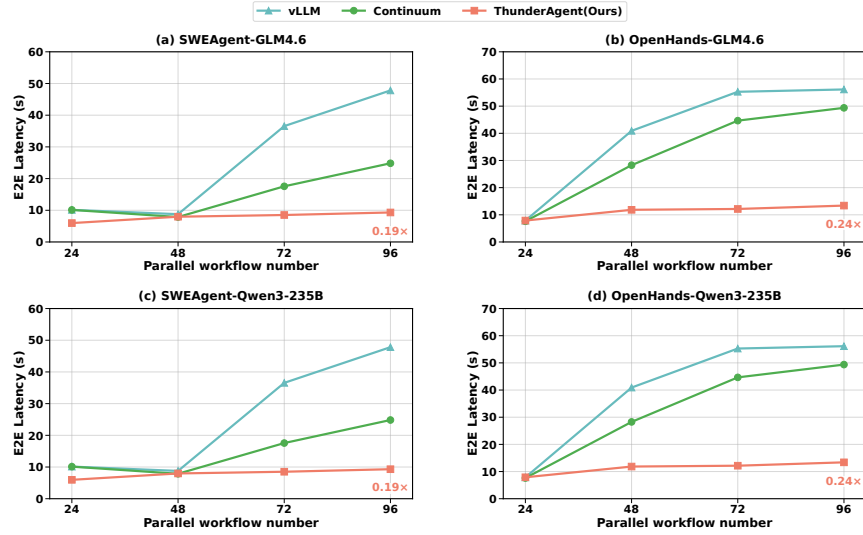


图 10: 端到端延迟比较